16 位流水线 CPU 核心设计文档

默认 AC1　CPU v1.6　ISA v1.5　架构版 r3　2026 年 11 月 2 日

本规范从指令契约、数据与状态边界、流水控制和系统验证四个层次描述一颗 16 位顺序单发射五级 CPU。datapath_core 沿用历史名称，实际包含数据通路和硬连线流水控制。设计目标是让每条指令的数据、控制意图、有效标签及副作用形成可检查的完整链条。

所有被接受且未取消的指令沿 IF→ID→EX→MEM→WB 传递。RF 和 flags 在 WB 写入；STR 在 MEM 写 DM 一次，并继续保留有效 WB 完成标签。Stall 延长停留，bubble 以 valid=0 表示，分支和故障按年龄取消年轻指令。五级并行工作，与全局五个运行模式不是同一概念。

默认生产实现固定为已验证 AC1。one-hot 前递和 WB 到 MEM 为两个隔离实验，后者使用自己的停顿契约；本架构整理不更改 ISA、生产 RTL 或周期。RF 影子、停止现场及 VIO 输入探针只做观测；VIO 的独立复位输出则经板级同步链控制 core_reset，须与被动观测区分。

课程目标为 EES-331、xc7z020clg484-1、M19/100MHz。离线功能与布局布线结果按证据范围列出；实板验收由用户延期，目标对应未确认的尝试不计 CPU 验证通过。

RF 为寄存器组，IM/DM 为指令/数据存储，CU 为控制职责，CW 为随行操作意图。数据宽 16 位，指令定长 16 位，地址以 16 位字为单位；机器字与寄存器值默认十六进制。

目录

1 设计目标与指令契约 4
1.1 设计目标与实现边界 4
1.2 指令长度与字段划分 4
1.3 指令编码与完整语义 5
1.4 立即数与地址规则 7
1.5 编码示例 7
1.6 核心执行接口 8
1.7 同步存储边沿契约 9
1.8 设计依赖关系与课程开发流程 9
2 数据通路与状态边界 11
2.1 执行核心与全局协作 11
2.1.1 整体控制与数据通路架构 12
2.1.2 五级数据通路详细连接 13
2.2 关键部件如何组成执行核心 14
2.2.1 寄存器组与立即数扩展 15
2.2.2 多路选择器 15
2.2.3 ALU 与 FPGA 进位链加减核心 16
2.2.4 进位溢出与标志寄存器 17
2.3 架构状态与观察状态的唯一性 17
2.4 四个流水边界的分类载荷 18
2.5 统一五级执行契约 19
2.6 典型指令的数据路径 20
2.6.1 ADD 立即数 20
2.6.2 LDR 与 STR 21
2.6.3 CMP 与 JZ 21
3 流水控制与跨级协调 23
3.1 控制模型与职责所有权 23
3.1.1 控制逻辑模型 23
3.1.2 内部功能与信号归属 24
3.1.3 本拍控制与时钟边沿 25
3.2 随行意图 载荷与当拍动作 26
3.2.1 ID 译码与指令操作意图 26
3.2.2 后级将意图变为实际动作 28
3.3 前递与流水协调的组成 29
3.4 数据相关与 load use 停顿 30
3.5 冒险分类与解决矩阵 30

3.6 写入资格与事件优先级 32
3.7 HALT 故障与复位 33
3.8 全局运行模式 FSM 33
3.8.1 运行模式与事件定义 33
3.8.2 状态转移与当前输出 34
3.8.3 排空完成与复位返回 35
3.9 五个实例贯通数据 CW 与当拍动作 36
3.9.1 相邻 ADD 的 M 到 EX 前递 36
3.9.2 默认 AC1 的 load use 保持与 bubble 37
3.9.3 CMP 到 JZ 的 Z 前递与年轻取消 37
3.9.4 STR 在 MEM 写入并在 HALT 前完成 WB 标签 38
3.9.5 较老 MEM 故障胜过年轻 EX 分支 39
4 系统集成与验证结果 40
4.1 系统层次与启动观测 40
4.1.1 文件初始化与系统连接 40
4.1.2 稳定停止与公开结果接口 41
4.1.3 实际结果与验收边界 42
4.2 板级时钟复位与观测边界 43
4.2.1 时钟复位与观测连接 43
4.2.2 完整实现与时序覆盖 44
4.3 源码边界与连接约定 44
4.4 数据通路与控制的 RTL 连接 46
4.5 分层验证方法与证据范围 47
4.5.1 自检查测试方法 47
4.5.2 既有实现与验证证据 48
4.5.3 统一五级与冒险验证结果 50
4.5.4 覆盖范围与集成验收 51
4.5.5 CPU 功能验证程序 56
4.6 性能基线与统计口径 57
4.6.1 CPU 性能验证 57
4.6.2 固定程序与性能基线 58
4.7 隔离性能候选与取舍 58
4.7.1 前递 MUX 组合候选 59
4.7.2 WB 到 MEM 存储数据候选 59
4.8 重现入口与验收边界 60
4.9 参考资料 61

1 设计目标与指令契约

先定义可观察的指令行为、接口和设计依赖。数据通路、译码、控制和 TB 都必须实现同一版本的 ISA，不能混用早期 Pre4 映射。

1.1 设计目标与实现边界

表 1 架构规格

项目	规定
数据与指令	数据 16 位 指令定长 16 位
通用寄存器	R0 至 R3 共 4 个 16 位寄存器；R0 普通可写
标志状态	Z 为等值比较结果；C 为算术进位或无借位；V 为有符号溢出
执行组织	IF ID EX MEM WB；顺序单发射 单时钟域
地址单位	一个地址对应一个 16 位字；顺序 PC 加 1
存储组织	Harvard 结构；IM 与 DM 独立；基线各 256 字
算术结果	保留低 16 位；按模 65536 计算
复位	高有效同步复位；PC RF Z/C/V 流水 valid 停止状态清 0

程序可观察的行为由寄存器、标志、控制流和 DM 内容决定。PC_fetch 是前端将要取指的地址，不等于最后执行或完成指令的地址；每条流水指令携带自己的 pc 标签，分支与错误定位使用该标签。

DM 不随核心复位逐字清空，初值由初始化镜像或程序确定。核心没有缓存、动态调度、ROB、中断返回或软件可恢复异常；停止保护仍须确保错误路径不写入状态。

1.2 指令长度与字段划分

表 2 指令字段

字段	位范围	位宽	用途
Opcode	[15:12]	4	选择 CPU 指令
Rd	[11:10]	2	目的寄存器或第一源；STR 的数据源
Rs	[9:8]	2	第二源或访存基址
imm8	[7:0]	8	无符号常数或有符号位移

```
word = (Opcode << 12) | (Rd << 10) | (Rs << 8) | imm8
```

寄存器形式的 imm8、立即数形式中未使用的 Rs，以及分支/HALT 未使用的寄存器字段，在程序镜像中置 0；硬件忽略未使用字段。编码工具须检查字段范围，负位移先编码为 8 位补码。bubble 用 valid=0 表示，不能用 MOV R0,#0 代替，因为该指令会改写 R0。

编码表中的 x 须按字段解释：imm8 中的 x 表示由程序填写的有效数据位；未使用 Rs 或分支/HALT 寄存器字段中的 x 表示该字段不影响执行。横线也表示未使用位。它们都不是要求写入 HDL 未知值 X；程序生成器统一把未使用字段置 0，硬件则接受这些字段的其他确定二进制取值。

表 3 各类指令的字段使用规则

指令形式	Opcode	Rd 字段	Rs 字段	imm8 字段
寄存器形式	0 2 3 4 6 A	目的或第一源	第二源	未使用 置 0
立即数形式	1 5 7	目的及需要时的旧值源	未使用 置 0	无符号常数
内存形式	8 9	LDR 目的或 STR 数据源	地址基址	有符号位移
分支形式	C D	未使用 置 0	未使用 置 0	C 绝对 u8 D 相对 s8
停止形式	E	未使用 置 0	未使用 置 0	未使用 置 0

保留编码 B/F 无数据操作数，产生非法指令标记。该 4+2+2+8 格式采用当前小组 ISA；课程资料中的 5 位 Opcode 扩展示例是另一种格式，不与本表拼接。

1.3 指令编码与完整语义

下面 d、s 分别是 Rd 和 Rs 索引；u8 范围为 0 至 255，s8 范围为−128 至 127；pc 指本条指令的 pc 标签。标志列中的“保持”表示不写对应架构状态。

表 4 指令编码与架构语义

Opcode	指令	操作数来源与结果去向	标志
0000 0	MOV Rd,Rs	R[d] ← R[s]	保持
0001 1	MOV Rd,#u8	R[d] ← ZExt(imm8)	保持
0010 2	AND Rd,Rs	R[d] ← R[d] AND R[s]	保持
0011 3	OR Rd,Rs	R[d] ← R[d] OR R[s]	保持
0100 4	ADD Rd,Rs	R[d] ← low16(R[d]+R[s])	更新 C/V

Opcode	指令	操作数来源与结果去向	标志
0101 5	ADD Rd,#u8	R[d] ← low16(R[d]+ZExt(imm8))	更新 C/V
0110 6	SUB Rd,Rs	R[d] ← low16(R[d]−R[s])	更新 C/V
0111 7	SUB Rd,#u8	R[d] ← low16(R[d]−ZExt(imm8))	更新 C/V
1000 8	LDR Rd,[Rs+s8]	EA=low16(R[s]+SExt(imm8))；R[d] ← DM[EA]	保持
1001 9	STR Rd,[Rs+s8]	同上生成 EA；DM[EA] ← R[d]	保持
1010 A	CMP Rs1,Rs2	Rs1 占 Rd 字段，Rs2 占 Rs 字段；Z ← (R[d]==R[s])；不写 RF	更新 Z
1011 B	Reserved	非法指令；不产生数据写入	保持
1100 C	JUMP #u8	下一执行目标=ZExt(imm8)	保持
1101 D	JZ #s8	Z=1 时目标=low16(pc+1+SExt(imm8))	保持
1110 E	HALT	取消年轻指令；老指令完成后停止	保持
1111 F	Reserved	非法指令；不产生数据写入	保持

ADD/SUB 不更新 Z，因此 JZ 始终使用最近一次有效 CMP 的结果。SUB 立即数为无符号 8 位常数，与 ADD 立即数一致。ISA 目前没有 ADDC、SUBB、JC 或 JO；保存的 C/V 可供测试或调试观察，尚无软件读取指令。

七组分类按控制副作用组织：Group 1 为 0 至 3，写 RF 但保持 Z/C/V；Group 2 为 4 至 7，写 RF 与 C/V 但保持 Z；Group 3 为 8/9，仅 LDR 写 RF，STR 写 DM，二者均保持标志；Group 4 为 A，仅写 Z；Group 5 为 C/D，控制 PC 而不写 RF/DM/标志；Group 6 为 E，发出停止请求；Group 7 为 B/F，产生非法指令标记而不产生数据写入。分组用于解释控制，不替代逐条指令译码。各组写入信号是意图，仍须按 valid、故障、取消与复位资格在规定流水级生效。

表 5 当前 ISA 与 Pre4 已验证映射的对应关系

指令	本版 v1.5 Opcode	Pre4 v1.4 Opcode	迁移要求
AND	2	8	两实际寄存器源，alu_op=3；只写 RF
OR	3	9	两实际寄存器源，alu_op=4；只写 RF
LDR	8	2	Rs 加符号位移；MR/RW=1，wb_sel=1
STR	9	3	独立基址与数据源；MW=1，RW=0

其余 12 个 Opcode 的含义不变，包括 B/F 保留编码。新旧机器字不能混用：v1.4 的 3300 是 STR，本版则是 OR；v1.4 的 2B00 是 LDR，本版则是 AND。迁移时重编译指令镜像，并检查译码、参考模型和逐拍观测属于同一版本。

MOV 及普通 ALU 指令在 WB 写 RF；LDR 在 MEM 读 DM、在 WB 写 RF；成功 STR 在 MEM 写 DM 一次，随后有效标签经过 WB 但不再写入；CMP 在 WB 写 Z。故障 STR 不写 DM，仍保留有效故障标记到 WB。JUMP/JZ 在 EX 重定向；HALT 在 EX 确认停止请求，在 WB 消费停止标记后完成停止。

1.4 立即数与地址规则

```
ZExt(imm8) = {8'h00, imm8}
SExt(imm8) = {{8{imm8[7]}}, imm8}
```

MOV/ADD/SUB 立即数及 JUMP 使用零扩展；LDR/STR/JZ 使用符号扩展。以 imm8=FF 为例，零扩展是 00FF，符号扩展是 FFFF。JZ 偏移以本条 pc+1 为基准，不以已经超前的 PC_fetch 计算。

有效地址先截为 16 位，再检查物理容量。256 字存储的合法地址为 0000 至 00FF；EA=0100 和 FFFF 均越界，访问被阻断。不能把 0100 只接低 8 位而访问 0000。0020+FFFF 得到 001F，表示基址 0020 加−1；该运算的 carry 不代表地址出错。

FFFF+1 回绕 0000 符合本规格的模地址语义。若改为禁止 16 位回绕，需要另定义扩宽地址规则与对应测试。指令取址越界携带 fetch_fault，数据越界在物理 DM 使能前检测；两者都按第三部分的年龄规则终止。

1.5 编码示例

表 6 机器字编码示例

汇编	Opcode	Rd	Rs	imm8	机器字
MOV R0,R1	0000	00	01	00000000	0100
MOV R0,#0x22	0001	00	00	00100010	1022
AND R0,R1	0010	00	01	00000000	2100
OR R0,R1	0011	00	01	00000000	3100
ADD R0,R1	0100	00	01	00000000	4100
ADD R0,#0x22	0101	00	00	00100010	5022
SUB R0,R1	0110	00	01	00000000	6100
SUB R0,#3	0111	00	00	00000011	7003
LDR R0,[R1+2]	1000	00	01	00000010	8102
STR R0,[R1+2]	1001	00	01	00000010	9102
CMP R0,R1	1010	00	01	00000000	A100

汇编	Opcode	Rd	Rs	imm8	机器字
JUMP #0x20	1100	00	00	00100000	C020
JZ −2	1101	00	00	11111110	D0FE
HALT	1110	00	00	00000000	E000
STR R1,[R2+4]	1001	01	10	00000100	9604

机器字按字段计算并独立校核。确认的编码表右侧有八项示例与字段不符：MOV/AND/OR/ADD/SUB 的 R0,R1 形式以及 CMP R0,R1 都须令 Rs=01，分别为 0100/2100/3100/4100/6100/A100；LDR R0,[R1+2] 为 8102，STR R0,[R1+2]为 9102。这些修正不改变其 Opcode 或汇编语义。原表的 8802/9802 实际选择 Rd=R2、Rs=R0，不能解释成 R0 与 R1。

JZ −2 的 DFFE 不是功能错误：其未使用 Rd/Rs 均为 11，按本规格仍忽略这些字段，offset=FE。程序生成器采用未使用字段为 0 的规范形式 D0FE。最终 PPT 第 5 页 STR R1,[R2+4]的 3604 属于旧映射，本版应为 9604。保存的原 PPT、源码和波形保留历史身份，后续程序镜像仅使用本版统一编码。

1.6 核心执行接口

方向以 CPU 核心为参考。clk/rst 是控制输入；各存储接口在同一时钟域工作，不含 ready 或可变等待协议。

表 7 CPU 核心执行接口

信号	方向	位宽	功能与时序
clk	输入	1	唯一核心时钟 上升沿更新状态
rst	输入	1	高有效同步复位 优先级最高
im_addr	输出	16	取指字地址 来自 PC_fetch
im_en	输出	1	合格取指请求；stall/停止/取消时阻止错误取指
im_dout	输入	16	同步 IM 输出；配合 IF/ID 的 pc/valid 供 ID 使用
dm_addr	输出	16	MEM 的完整 16 位 EA；物理截位前检查范围
dm_en	输出	1	合格 MR 或 MW 访问使能
dm_we	输出	1	合格 STR 写使能；仅 MEM 有效访问时为 1
dm_din	输出	16	EX/MEM 保存的已转发 STR 数据
dm_dout	输入	16	同步 DM 读结果；供 WB 的 load 选择
halted	输出	1	HALT 完成排空后的停止状态 直到复位

信号	方向	位宽	功能与时序
fault	输出	1	故障完成排空后的停止状态 直到复位

表 7 列出原 12 项执行接口，合计 87 位；本版追加的只读观测接口另见第 4.1 节及系统集成表，不计入此 87 位。逻辑端口不等于 FPGA 外部引脚；cpu_system 把执行接口接至片上 IM/DM，无需无协议转换职责的 Adapter。

正常取指接受条件为未复位、处于 RUN、没有 load use 保持且没有本拍更高优先级的取消事件。im_en 还须满足 im_addr 在物理范围内。取址越界时禁止实际 IM 访问，同时把该 pc 与 fetch_fault=1 送入有效 IF/ID 槽；译码器仍可组合译码所保持的 IM 输出，但故障资格阻止其作为有效指令执行。该错误可被更老分支取消，否则传到 EX 确认停止。

1.7 同步存储边沿契约

表 8 同步存储读时序

周期位置	IM 路径	DM 读路径
周期 k 内	地址与 im_en 稳定	MEM 的 EA 与读使能稳定
k 结束上升沿	采样地址 同步登记 pc/valid	采样地址 同步登记 rd/写回控制
周期 k+1 内	dout 供 ID 译码 与 IF/ID 标签一致	dout 供 WB 选择 与 MEM/WB 标签一致
k+1 结束上升沿	ID/EX 捕获译码及源值	RF 捕获当前 WB 的 load 数据

IM 在 en=0 时保持已返回指令，使 load use 停顿中的 ID 载荷稳定。同步存储的输出寄存器可以承担相应逻辑流水边界；同一边界不要无意再串接同沿 IR/MDR 而增加一拍。实际 IP 的读延迟、输出寄存器、en 保持行为和写模式须与该契约对应。

DM 写入在 MEM 结束上升沿发生。背靠背 STR 后同地址 LDR 必须读到已写的新值；本基线不会在同一个单端口周期同时请求读和写。IM 与 DM 独立，允许 IF 取指与 MEM 访存同周期发生。

1.8 设计依赖关系与课程开发流程

设计依赖顺序为：明确任务和接口约束 → 确定 ISA 语义 → 列出指令微操作与副作用 → 分配资源和流水级 → 定义级间载荷 → 设计译码、局部控制与跨级协调 → RTL 和分层验证 → 系统集成 → 综合、布局布线及性能比较 → 实物验收。后一级设计必须能解释前一级契约如何生效。

项目实际采用课程逐步集成的方法：Pre2 先完成 mux 和加法器，Pre3 补齐 ISA 及其他关键模块，Pre4 连接数据通路，Pre5 整理流水 CU，随后完成文件启动系统、板级离线实现和隔离性能实验。本规范将成熟实现按整机依赖关系组织，不把实际开发时间线改写为一开始已完成全部设计。

表 9 课程任务与本规范的对应关系

课程阶段	应说明的成果	主要阅读位置
Pre2	mux、加法器及人工/EDA 验证	第二章部件契约；第四章验证方法及历史原始记录
Pre3	ISA、RF、ALU、PC、立即数扩展及验证	第一章指令契约；第二章资源和状态；第四章覆盖
Pre4	数据通路、接口、典型指令、源码与波形	第二章五级路径和边界；第三章逐拍例；第四章连接与证据
Pre5	CU 结构、工作原理、状态图	第三章控制层次、CW、仲裁、FSM 与并行实例
Pre6	设计流程、整机集成、功能和性能	本节设计依赖；第四章系统层次、功能和性能结果

2 数据通路与状态边界

本章说明值从哪里来、何时就绪、存在哪里。控制职责用于说明连接，详细动作和仲裁在第三章展开。

2.1 执行核心与全局协作

CPU 执行核心由数据通路与硬连线流水控制共同组成，RTL 顶层沿用历史名称 datapath_core。全局 mode 管理运行、排空和停止；跨级协调处理相关与年龄优先级；局部 CU 把本级 CW 和资格转换为本拍 DP 选择和状态更新。RUN 可以连续保持很多拍，期间五级同时处理不同指令。

下表是一页总体契约。图中 CU 方框是逻辑职责，u_*才是实际实例；CW_X/M/W 是现有字段组。实现没有要求一个方框对应一个源文件，也没有五个各自运行 IF→WB 循环的局部 FSM。完整所有权见第 3.1 节，载荷分类见第 2.4 节。

表 10 整机控制与数据的总体契约

层次/对象	提供的信息或资源	边界与完成规则
全局模式 FSM	RUN、两种 DRAIN、HALTED、FAULTED	提供运行资格并接收合格事件；不直接包办所有级使能
跨级协调	源/目的、valid/fault、就绪、分支与停止事件	统一决定最近生产者、前端保持、年轻取消和排空
五级局部职责	IF 请求；ID 译码/发射；EX 运算/决定；MEM 保护；WB 写入	意图随标签传递；实际动作在规定阶段由本拍资格形成
DP 与架构状态	唯一 PC、RF、flags、MUX、ALU 和目标计算	RF/flags 在 WB 写；STR 在 MEM 写一次；所有存活指令标签继续到 WB
四个逻辑边界	同一指令的数据、CW 和元数据	IM/DM 寄存输出承担两端数据边界；不额外串接 IR/MDR
系统及板级观察	同步 IM/DM、文件启动、RF 影子、停止现场、VIO 和计数	只读探针不反馈；VIO 复位输出独立请求 Reset，Reset 保留 DM；物理验收延期

2.1.1 整体控制与数据通路架构

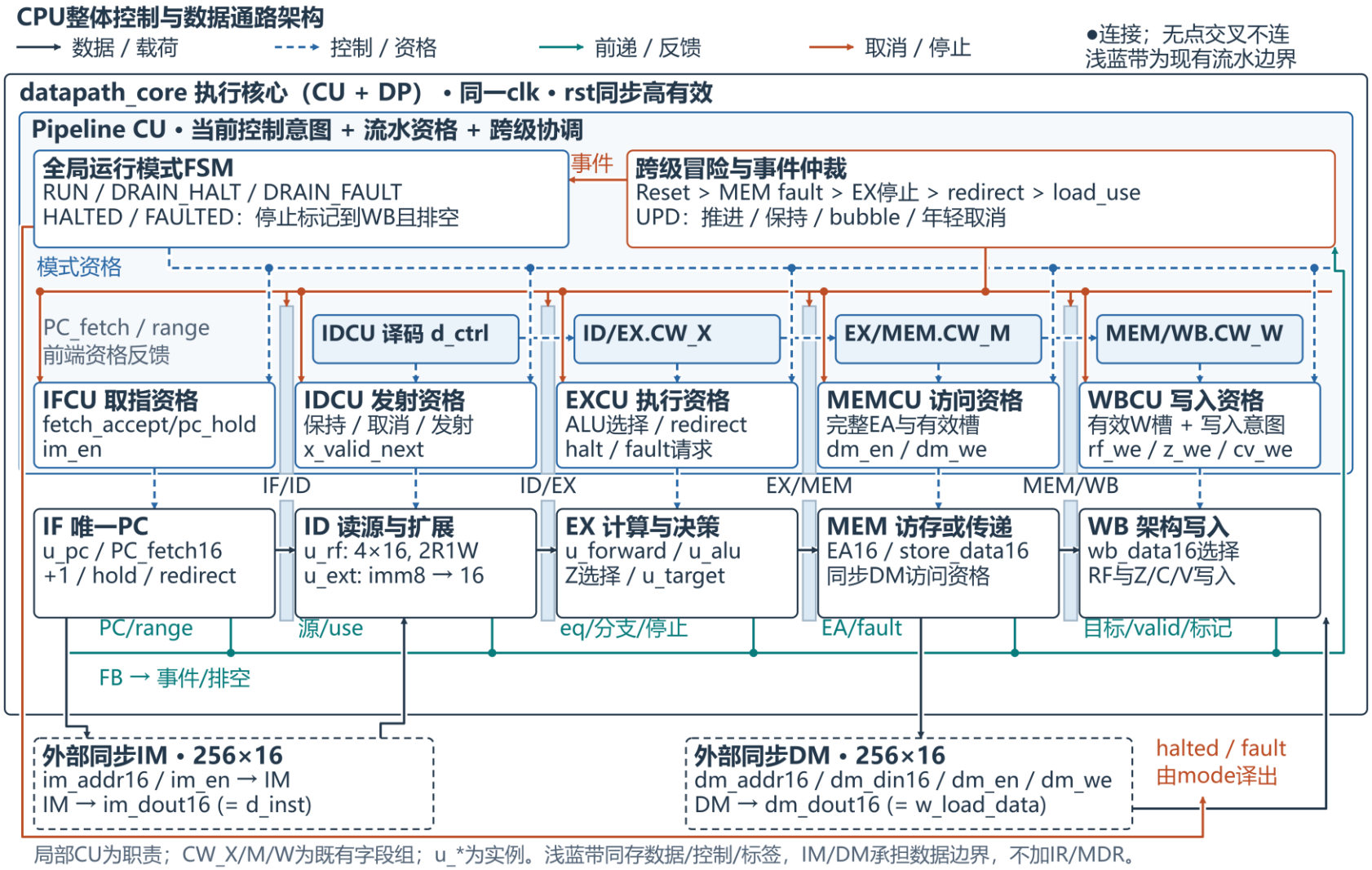


图 1 整体控制与数据通路架构。随行控制先输入各级控制逻辑，再与运行模式、有效资格及跨级仲裁共同产生使能。浅蓝边界同时关联数据、标签与控制；IFCU 至 WBCU 为现有职责划分，对应见表 20。

2.1.2 五级数据通路详细连接

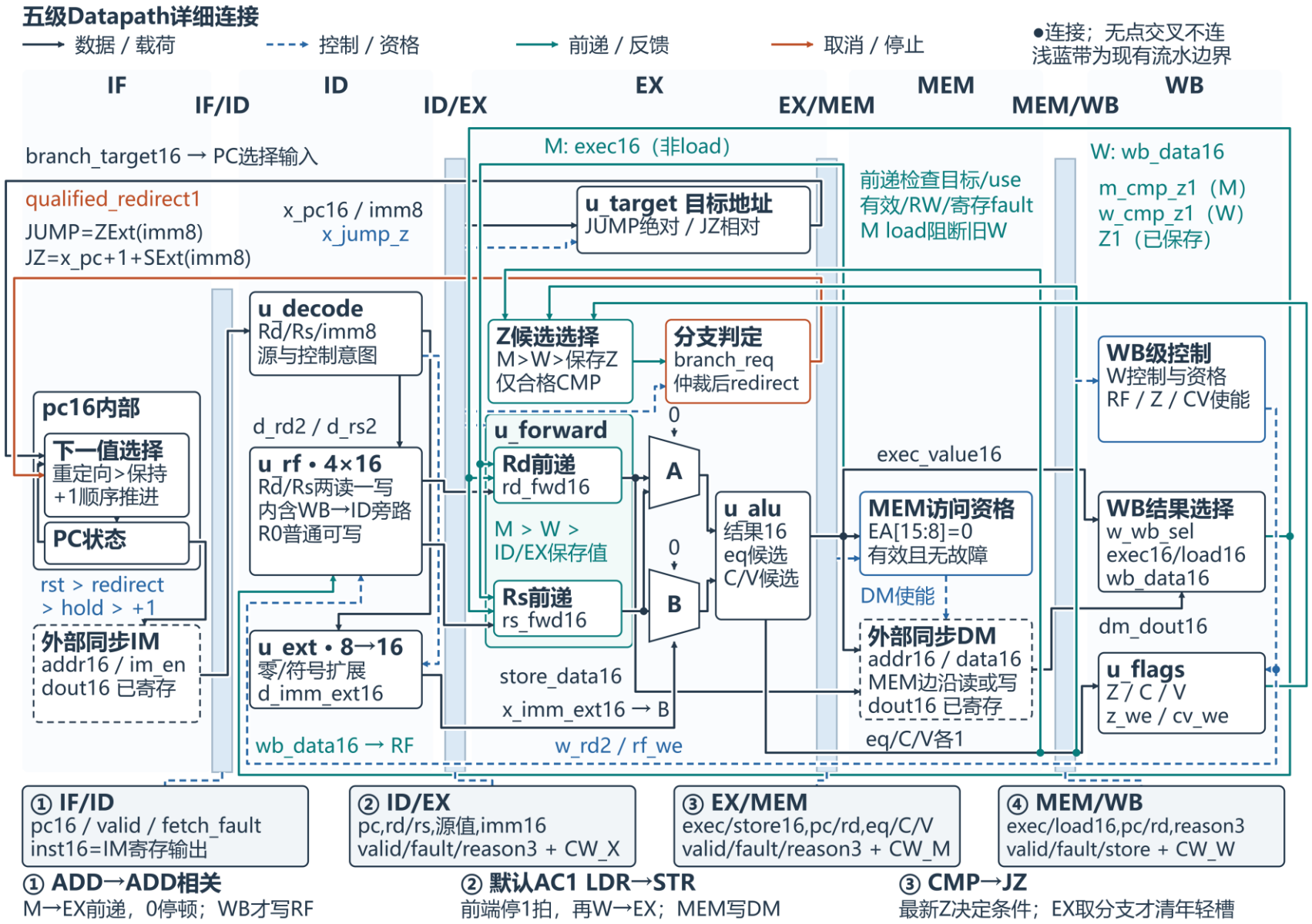


图 2 五级 Datapath 详细连接。目标计算与分支判定分开；M/W 前递、RF 写回及 Z 反馈均标出源端。浅蓝带为既有流水边界，完整字段见表 15；同步存储输出承担相应数据边界。

图 1 展示控制意图经各级逻辑形成合格使能的过程；图 2 按实际连线追踪数据。相邻 ADD→ADD 相关由 M 非 load 结果前递到 EX，最终仍在 WB 写 RF；LDR→STR 相关保持前端一拍，再从 W 前递，STR 由 Rd 提供写数据、Rs 加立即数形成地址；CMP→JZ 由最近合格比较候选提供 Z，Z 只参与分支条件，取分支才取消年轻槽，u_target 独立计算目标。rd_fwd/rs_fwd 的默认源是 ID/EX 保存值；WB 数据与 RF 写地址/使能分别返回，RF 内部完成同址读旁路。

ALU 结果与有效地址共用 exec_value，因为一条指令不会同时需要二者；STR 独立保存已前递 Rd 为 store_data，Rs 用于基址。IM 寄存输出就是 D 级指令数据，DM 寄存输出就是 W 级加载数据；不再串接 IR/MDR。load_use 只保持 PC/ID 并向 EX 插入 bubble，后端继续。分支在 EX 重定向但自身继续到 WB；STR 在 MEM 仅写一次，WB 有效而不写入。HALT/fault 标记排空至 WB，Reset 取消在途载荷并禁止当拍写入。

默认 AC1 的 u_forward 合并 CU 的相关比较/优先控制和 DP 的 16 位 MUX。WB 到 MEM 另列实验。图上分层不增寄存器，load16 取同步 DM 的 dout；完整字段见表 15。

2.2 关键部件如何组成执行核心

表 11 主要模块接口与职责

模块	输入	输出	职责
isa_decode	inst[15:0]	rd/rs imm 控制字 illegal	将 4 位 Opcode 映射为内部选择及副作用意图
regfile4x16	clk rst we 两读地址及写地址各 2 位 wdata16	两路读值各 16 位	4×16 双读单写 含 WB 到 ID 旁路
imm_ext16	imm[7:0] sign_mode	value[15:0]	明确零扩展或符号扩展
operand_forward	实际源索引/use 保存值 M/W 生产者	rd_fwd16 rs_fwd16	为 EX 选择最新已就绪操作数
alu16	a16 b16 op3	result16 eq illegal carry overflow	组合运算 独立等值比较 产生 C/V 候选
pc16	clk rst hold redirect target16	pc16	复位 重定向 保持 顺序加 1
branch_target16	branch_pc16 imm8 relative	target16	JUMP 绝对目标或 JZ 相对目标
flags_reg	clk rst z_we cv_we z_in c_in v_in	Z C V	按掩码保存标志

模块	输入	输出	职责
hazard_control	各级 valid 实际源/use load 目标 停止及重定向事件	hold kill 取指资格	仲裁停顿 冲刷和排空
pipe_regs	clk rst 各级控制及载荷	四组流水载荷	保存数据 控制和有效性

当前 AC1 包含原 carry/overflow 接口、flags_reg 及流水控制，复用 ISA v1.5 译码、镜像和 TB。加减核心的物理映射与冻结 CLA 不同，但算术、标志和流水契约不变。表中的 hazard_control 和 pipe_regs 是 datapath_core.sv 内的逻辑职责，未新增同名模块；实体仍按原命名端口及位宽连接。

2.2.1 寄存器组与立即数扩展

RF 写端在上升沿更新；rst 清空四个寄存器，否则 we 选择唯一写地址，其他寄存器保持。两读端为组合读，读地址分别接 Rd 和 Rs；we 有效且未复位时，同地址读端旁路当前 WB 的 wdata，保证 ID 捕获本拍写回值。该旁路不依赖仿真进程先后，也不采用 R0 常量零特例。

use_rd/use_rs 只描述实际源读取。MOV 立即数不读旧 Rd，LDR 只读 Rs，STR 同时读 Rd 数据和 Rs 基址。ID 仍可让两个读口输出值，但未使用源不能触发相关停顿。

扩展器无状态。sign_mode=0 时高 8 位为 0，sign_mode=1 时复制 imm8[7]。扩展结果与本条译码控制一起进入 ID/EX，之后不使用当前 ID 指令的模式控制较老 EX 载荷。

2.2.2 多路选择器

表 12 数据通路选择编码

选择点	编码	输出用途
EX 转发选择	M 结果优先 W 结果次之 最后保存值	分别形成 rd_fwd 和 rs_fwd
alu_a_sel[1:0]	0=常数 0 1=rd_fwd 2=rs_fwd 3=保留	ALU a
alu_b_sel[1:0]	0=常数 0 1=rs_fwd 2=imm_ext 3=保留	ALU b
ALU result 选择	PASS_B ADD SUB AND OR	result16
wb_sel	0=exec_value 1=load_data	RF 写回 wdata
Z 有效值选择	M 级 ZW 候选 W 级 ZW 候选 架构 Z	JZ 条件
PC 下一值	合格 redirect 目标 否则 hold 或 pc+1	PC_fetch

MUX 是组合选择逻辑，不保存数据。先对 Rd/Rs 完成转发，再选择 ALU 输入。STR 将 rd_fwd 直接保存为 store_data，不把它误接为地址加法 a 输入。

人工设计的单比特 2 选 1 关系为 y=(NOT s AND d0) OR (s AND d1)，16 位选择器对每一位重复该关系。行为 HDL 可用条件运算符或完整 case 实现，结构 HDL 可用门电路或小 MUX 实例化；二者必须具有相同选择编码。保留选择码使用安全默认值，并由有效控制检查报告非法情况。

2.2.3 ALU 与 FPGA 进位链加减核心

ALU 内部 op 不是 CPU Opcode。0=PASS_B，1=ADD，2=SUB，3=AND，4=OR；5 至 7 返回 result=0、illegal=1、carry=0、overflow=0。eq 始终等于 a==b，独立于 op；只有 CMP 的 ZW 允许其最终更新 Z。

ADD/SUB 仍共用 addsub16_cla 接口，sub=0 执行加法，sub=1 以 a 加 b 逐位取反再加 1 实现减法。AC1 在无符号 17 位表达式中保留第 16 位进位，低 16 位为结果；saved C 不作为输入，未新增算术周期。

```
b_mod = b ^ {16{sub}}
wide_sum = {1'b0,a} + {1'b0,b_mod} + {16'b0,sub}
{carry, result} = wide_sum;  // carry=wide_sum[16], result=wide_sum[15:0]
```

AC1 运算层级为 datapath_core/u_alu → alu16/u_addsub → addsub16_cla 内 17 位表达式；该路径不实例化 cla16 或 cla4_pg。以下 P/G 与 K 公式仅说明冻结 v1.6 两层 CLA 及保留的独立 CLA 单元：cla16 包含四个 cla4_pg 及组间超前网络，p=a XOR b、g=a AND b、c0=cin。

历史 CLA 说明采用 XOR 传播；课件也用 p=a OR b，配合 g=a AND b 可得到等价进位函数，但不能将 OR 传播量代入 sum=p XOR c。以下 CLA4 公式保留供独立单元核对，不是 AC1 的综合算术实现。

```
sum[i] = p[i] ^ c[i]
c1 = g0 | (p0 & c0)
c2 = g1 | (p1 & g0) | (p1 & p0 & c0)
c3 = g2 | (p2 & g1) | (p2 & p1 & g0) | (p2 & p1 & p0 & c0)
P = p3 & p2 & p1 & p0
G = g3 | (p3 & g2) | (p3 & p2 & g1) | (p3 & p2 & p1 & g0)
c4 = G | (P & c0)
```

四组 P/G 记为 P0 至 P3、G0 至 G3。K0=cin，K1 至 K4 分别为 c4/c8/c12/c16；每组入口由顶层同时求得：

```
K1 = G0 | (P0 & K0)
K2 = G1 | (P1 & G0) | (P1 & P0 & K0)
K3 = G2 | (P2 & G1) | (P2 & P1 & G0) | (P2 & P1 & P0 & K0)
K4 = G3 | (P3 & G2) | (P3 & P2 & G1)
   | (P3 & P2 & P1 & G0) | (P3 & P2 & P1 & P0 & K0)
```

上列组 P/G 不依赖 cin、K1 至 K4 采用组间超前，这是冻结 v1.6 及独立 cla16 单元的结构要求。AC1 加减路径由 17 位表达式推断 CARRY4，旧模块名不能证明内部仍是 CLA；速度与资源依据相同条件的实际 OOC 报告，不能只凭 CARRY4 数量判断。

2.2.4 进位溢出与标志寄存器

AC1 的 carry 取 wide_sum[16]，等价于 16 位算术的 c16。ADD 时表示无符号精确和大于 65535；SUB 时定义为无借位，unsigned(a)≥unsigned(b)则为 1。若调试 borrow，SUB 下 borrow=NOT carry；overflow 与 Z 写入规则不变。

教材加减器还将 unsigned_overflow 定义为 carry XOR sub：ADD 时检测进位，SUB 时检测借位。它与本规格保存的原始 C 不是同一个信号；如展示该诊断量，应另命名 u_overflow 并注明解释。V 始终表示有符号溢出，不能将借位或最高位进位直接称为 V。

overflow 表示精确有符号运算超出−32768 至 32767，公式使用原始 b 的符号位：

```
ADD_V = ~(a[15] ^ b[15]) & (result[15] ^ a[15])
SUB_V =  (a[15] ^ b[15]) & (result[15] ^ a[15])
```

非算术操作的组合 C/V 输出为 0，但只在 CVW=1 时才更新保存的 C/V。CMP 更新 Z 并保持 C/V；LDR/STR 尽管用 ADD 算地址，也保持所有架构标志。flags_reg 的同步 rst 最高优先级，z_we 和 cv_we 分别写各自字段，同时有效时分别更新。

表 13 算术结果与进位溢出验收向量

运算	result	C	V	原因
FFFF+0001	0000	1	0	无符号越界 有符号−1+1 正常
7FFF+0001	8000	0	1	有符号正数越界
8000+8000	0000	1	1	两类越界同时发生
0005−0003	0002	1	0	无借位 无符号及有符号均正常
0000−0001	FFFF	0	0	借位 有符号结果−1 正常
8000−0001	7FFF	1	1	无借位 有符号负数越界
7FFF−FFFF	8000	0	1	借位 32767−(−1)越界
1234−1234	0000	1	0	无借位 等值减法

以上为验收预期值。result[15]表示结果位串的最高位，不能直接当 overflow；carry 与 overflow 分别检查。

2.3 架构状态与观察状态的唯一性

ID 读的 RF 与 WB 写的 RF 是同一份 u_rf；EX 使用的已保存 Z 与 WB 更新的 Z 来自同一份 u_flags。M/W 中的 cmp_z、c 和 v 是尚未生效的候选数据，不是额外的架构 flags。RF 写回影子用于读出结果，不参与取数、前递、分支或任何控制判断。

硬件文件和逻辑职责允许交叉：operand_forward 既包含相关比较与优先选择，也包含 16 位数据 MUX；datapath_core 沿用历史名称，内部同时实现 DP 和硬连线流水控制。没有证据表明必须新增五个实体 CU 模块，文中的局部 CU 名称均表示职责。

表 14 状态对象和唯一更新责任

对象	实际保存位置	更新与使用规则
PC	u_pc.pc，16 位	唯一取指地址状态；Reset/redirect/hold/+1 按资格采样
架构 RF	u_rf.regs，4×16 位	ID 两读、WB 一写；内部 WB→ID 旁路；R0 普通可写
架构 flags	u_flags.Z/C/V	WB 按 z_we/cv_we 独立写；EX 只读取或选择较新 CMP 候选
D/X/M/W 载荷	core 现有流水字段及同步 IM/DM 输出	数据、CW 和标签属于同一条指令；一个字段只由其所属时序块更新
诊断原因与停止现场	core 的原因/观测时序块	reason 随标签推进；stop_pc/reason 在停止边沿锁存并保持至 Reset
RF 影子、最后 store	cpu_system 观察时序块	只采样合格写事件；无执行反馈，Reset 清观察状态但不清 DM
板级复位及计数	ees331_application	请求同步、释放去抖和公开事件计数；不新增 CPU 流水级

2.4 四个流水边界的分类载荷

数据、控制、pc 和 valid 属于同一条指令。下面列的是逻辑字段；同步 RAM 输出承担相应数据边界，不增加 IR/MDR。本版 X/M/W 各增加 3 位 reason 诊断字段，随同一有效标签推进；它不反馈控制，也不改变原指令行为。

表 15 数据 CW 与元数据的级间载荷

边界	数据	元数据与标签	CW	保存位置
IF/ID	inst16 由同步 IM 的 dout 保存	d_pc16、d_valid1、d_fetch_fault1	尚未译码，无 CW_X	core 标签 18 位 + 外部 IM 输出 16 位；不另加 IR
ID/EX	x_rd_value16、x_rs_value16、x_imm_ext16	x_pc16、x_rd2、x_rs2、x_valid1、x_fault1、x_reason3	CW_X 18 位逻辑集合	core 现有寄存字段 91 位；imm_signed 已在 ID 消费，不继续保存

边界	数据	元数据与标签	CW	保存位置
EX/MEM	m_exec_value16、m_store_data16、m_cmp_z1、m_c1、m_v1	m_pc16、m_rd2、m_valid1、m_fault1、m_reason3	CW_M 7 位逻辑集合	core 现有字段 65 位；exec 可作 ALU 结果或 EA，store 数据独立
MEM/WB	w_exec_value16、w_cmp_z1、w_c1、w_v1；load16 来自同步 DM 的 dout	w_pc16、w_rd2、w_valid1、w_fault1、w_reason3、w_store1	CW_W 5 位逻辑集合	core 字段 48 位 + 外部 DM 输出 16 位；不另加 MDR

imm_signed 在 ID 消费后无需继续保存；rs 和实际源掩码保留至 EX 完成转发，之后删除。STR 的 store_data 只保留到 MEM；新增 w_store 类型与 pc/valid 保留至 WB，RW/ZW/CVW 均为 0。HALT 及已确认的故障必须保留有效停止标记到 WB；没有 RF/标志写入不等于该槽可以提前丢弃。无效数据载荷可不复位，valid 与架构状态必须复位。kill 可用清 valid 实现，不强制每级另设 killed 寄存器。

2.5 统一五级执行契约

IF 统一取指并登记标签。所有接受且未取消的指令按 ID EX MEM WB 传递；无副作用的有效阶段与 valid=0 的 bubble 不同。停顿延长驻留时间，flush 取消年轻载荷，reset 取消全部在途载荷；五级组织不等于每条指令固定五拍。

表 16 各指令的有效阶段职责

编码	ID	EX	MEM	WB
0 MOV reg	读 Rs	PASS_B Rs	有效传递，无 DM	写 Rd，无 flags
1 MOV imm	无实际 RF 源；ZExt	PASS_B imm	有效传递，无 DM	写 Rd，无 flags
2 AND / 3 OR	读 Rd/Rs	两源前递，位逻辑	有效传递，无 DM	写 Rd，保持 flags
4 ADD / 6 SUB reg	读 Rd/Rs	加减 result/C/V	有效传递，无 DM	写 Rd/C/V，Z 保持
5 ADD / 7 SUB imm	读 Rd；ZExt	加减以 imm 为 b	有效传递，无 DM	同上
8 LDR	只读 Rs；SExt	基址前递，EA	合法 DM 同步读	load 写 Rd，flags 保持
9 STR	Rd 数据/Rs 基址；SExt	两源分别前递	合法 DM 写一次；valid 继续	store/pc/valid 保留，无写入
A CMP	读 Rd/Rs	eq 产生 cmp_z	cmp_z/ZW 有效传递	只写 Z

编码	ID	EX	MEM	WB
C JUMP / D JZ	无 RF 源；目标扩展	EX 合格 redirect；JZ 取最新 Z	分支自身有效传递，无 DM	有效完成，无写入
E HALT	停止意图	按年龄确认 DRAIN	停止 token 有效传递	消费 token 且下一拍空才停止
B/F 与 fetch fault	故障标签	未被取消则确认 fault	无副作用 token 传递	消费 token，FAULTED

STR 的副作用边沿保持 MEM，WB 只消费完成标签；同拍的另一个 M 指令可以合法访存，不能因为 W 是 STR 就全局禁止 DM。复位不回滚已完成的 DM 写入，本设计无 ROB 或异常返回。

2.6 典型指令的数据路径

以下算术、访存和比较分支用例分别说明 RF 写回、DM 访问以及 Z 与控制流的关系。相同规则也适用于其他采用同一路径的指令。

2.6.1 ADD 立即数

例：pc=0000，机器字 5001，初始 R0=7FFF、Z=1。目标是 R0=8000、C=0、V=1、Z 保持 1。

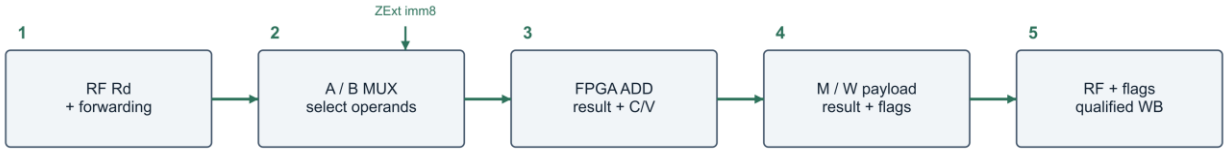


图 3 AC1 ADD 的数据流编号。①读取 Rd 并转发；②零扩展立即数经 B MUX；③17 位表达式映射 FPGA 进位链，生成低 16 位结果及 C/V；④候选值随流水传递；⑤WB 按原资格写 RF 和 C/V。

表 17 ADD 立即数逐拍操作

周期	阶段	数据与控制	结束边沿
C1	IF	im_addr=0000 合格 im_en=1	IM 返回 5001 登记 pc/valid
C2	ID	Opcode=5；use_rd=1 use_rs=0；imm_ext=0001；RW=1 CVW=1	保存 R0 旧值 源索引及控制
C3	EX	a=rd_fwd=7FFF b=0001 op=1；result=8000 C=0 V=1	EX/MEM 锁存 result 和 C/V 候选
C4	MEM	MR=MW=0；DM 不访问	载荷传入 MEM/WB
C5	WB	wb_sel=0；合格 RW/CVW 为 1；ZW=0	R0 写 8000 C/V 写 0/1 Z 保持

SUB 令 op=2，经同一 17 位补码加减表达式。MOV 选择 PASS_B；AND/OR 选择相应逻辑结果且 CVW=ZW=0，仍沿原普通写回路径。

2.6.2 LDR 与 STR

LDR 例：8BFF 表示 LDR R2,[R3−1]。设 R3=0020、DM[001F]=BEEF，地址流为 Rs → 转发 → A MUX，加 SExt(FF)=FFFF，得到 EA=001F；load 数据流为 DM → WB MUX → R2。

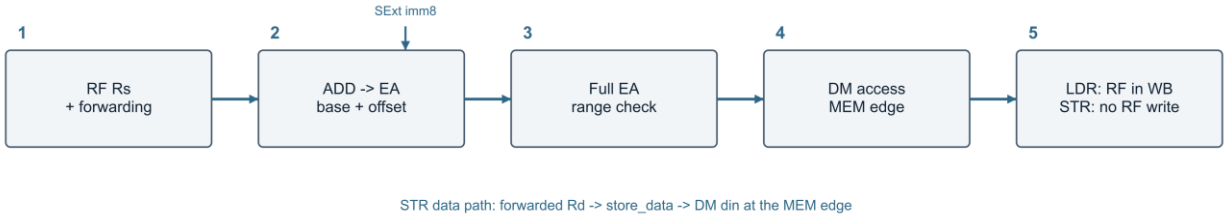


图 4 访存数据流。①转发 Rs 提供基址；②符号位移参与 ADD；③完整 EA 范围检查；④LDR 读取 DM 并选入 WB；⑤STR 另将转发 Rd 保存至 store_data 并在 MEM 写入。

表 18 LDR 与 STR 逐拍操作

周期	阶段	LDR 数据与控制	STR 数据与控制
C1	IF	取指并保存 pc	取指并保存 pc
C2	ID	use_rs=1；MR=RW=1 wb_sel=1；ZW=CVW=0	use_rd=use_rs=1；MW=1 RW=0；ZW=CVW=0
C3	EX	转发 Rs 加符号位移；EA 进 exec_value	同样生成 EA；rd_fwd 另锁入 store_data
C4	MEM	范围合法后 dm_en=1 dm_we=0；读结果随 rd/valid 供 WB	范围合法后 dm_en=dm_we=1；边沿写 DM
C5	WB	选 load_data，合格 RF 写入；EA 不写 RF	有效 STR 标签到 WB；不写 RF 或标志，不再次访问 DM

以 9900 为例，STR R2,[R1+0]将 R2 数据写入 R1 指定的 DM 地址。基址与数据各自转发，不能只转发地址而留下旧 store_data。load 数据在 MEM 结束后就绪，因此紧邻消费者须按下文停 1 拍。

2.6.3 CMP 与 JZ

例：pc0 装 A100，pc1 装 D001，pc2 装 1C55，pc3 装 E000；初始 R0=R1=0001。CMP 产生 1，JZ 目标=0001+1+0001=0003，跳过 pc2 的 MOV R3,#0x55。

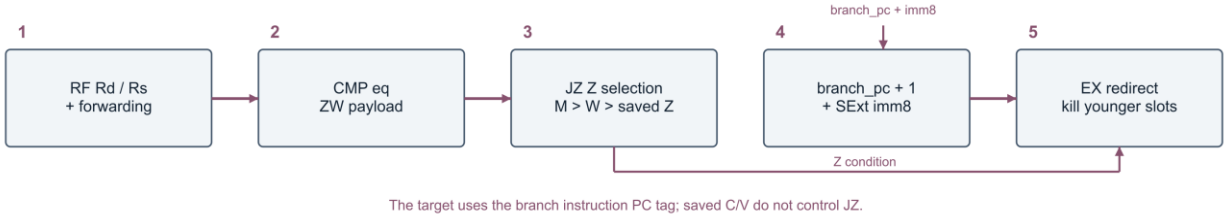


图 5 比较与分支数据流。①两个实际源转发至 eq；②CMP 结果随 ZW 进入 M/W；③JZ 选择最近较老 Z；④用分支自身 pc 计算目标；⑤EX 重定向 PC 并取消年轻槽。

表 19 CMP 与紧邻 JZ 逐拍操作

周期	CMP	JZ	前端与架构动作
C1	IF	未进入	取 pc0
C2	ID	IF	译码 CMP 并取 pc1
C3	EX 产生 cmp_z=1	ID	取 pc2 默认顺序推进
C4	MEM 携带 ZW 与 cmp_z	EX	JZ 由 M 级转发取得 Z=1；周期末 PC 改 0003，取消 ID 的 pc2 和当拍 IF
C5	WB 更新架构 Z=1	MEM 无数据写入	目标 pc3 进入 IF

JZ 无需等待 CMP 写回，但只从有效、无故障、未取消且 ZW=1 的生产者取 Z。M 优先 W 再架构 Z。若 M 是 ADD 而 W 是 CMP，仍须取 W 的 Z。候选 0 也必须覆盖旧 1。条件不满足时不冲刷，继续顺序取指。

JUMP 不读标志，目标为 ZExt(imm8)，仍在 EX 执行同样的重定向与年轻槽取消。没有 delay slot。

3 流水控制与跨级协调

本章从控制模型与所有权，进入 ID 意图及 CW 传播，再说明前递、保持、取消和停止。全局模式只管理运行资格，不能把五状态当成五级执行序列。

3.1 控制模型与职责所有权

3.1.1 控制逻辑模型

CU 采用硬连线控制，由当前指令和流水状态共同决定本拍动作。正常 RUN 中，五级可同时处理不同指令；ALU 为组合逻辑，IM/DM 读延迟固定。图 6 用次态逻辑 F、控制状态 Q 和输出逻辑 G 描述现有实现。

$$Q^{+} = F(Q, I) \quad O = G(Q, I)$$

F 根据当前状态与输入计算下一控制状态，Q 在时钟上升沿锁存，G 在当前周期产生控制输出。I 包括 d_inst、取指故障、ALU 非法标记、保存的 Z 与 M/W 比较候选、PC_fetch、MEM 有效地址和 rst；这些是逻辑输入集合，不是新增外部端口。

Q 包含 mode、各级 valid/fault、X/M/W 随行控制和寄存器标签；操作数、立即数、地址和结果数值由数据通路保存。G 产生操作数及前递选择、PC/IM 控制、DM 访问资格、RF 与标志写入资格，以及 halted/fault。F 和 G 可共享组合判定，逻辑划分不要求复制硬件。

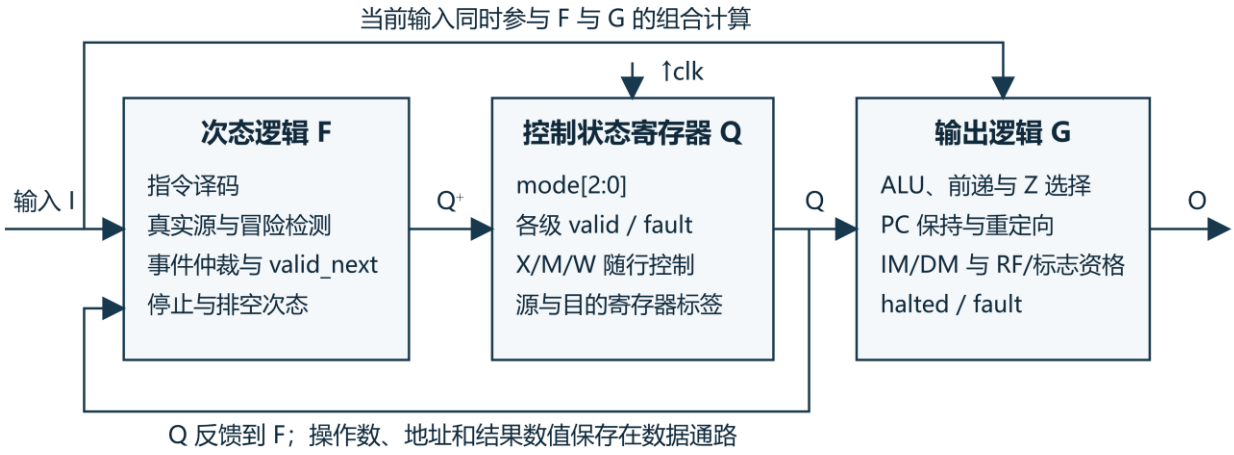


图 6 硬连线 CU 的 F/Q/G 逻辑模型。Q 反馈至 F，当前输入同时参与 F 和 G；图中方框表示现有控制职责。

3.1.2 内部功能与信号归属

表 20 控制所有权与 RTL 对应

职责/对象	信息或意图来源	仲裁与资格	实际驱动与边沿
全局运行模式	core 组合逻辑由当前 mode 及合格故障/HALT/drain_done 求 mode_next	Reset > MEM/EX fault > HALT；停止状态保持至 Reset	core 主时序块唯一更新 mode；非 IF 到 WB 的串行 FSM
指令译码	u_decode 产生源使用、ALU 选择和 RW/MR/MW/ZW/CVW 等意图；u_ext 在 ID 解释立即数	core 用 d_valid/d_fault 及发射资格屏蔽无效控制	译码为组合逻辑；意图在进入 ID/EX 的边沿锁存
IF 请求与 PC	EX 目标、mode、load_use 及事件共同提供取指资格	core 产生 qualified_redirect、fetch_accept、pc_hold 及 im_en	u_pc 唯一保存 PC；同步 IM 保存指令 dout；core 保存 D 标签
ID 保持与发射	ID 实际源与 EX load 目的比较形成 load_use	core 统一计算 d_valid_next/x_valid_next，非独立 ID 暂停 FSM	主时序块更新 D/X 有效位；停顿时 PC/IM/D 标签共同保持、X 注入 bubble
EX 操作数前递	u_forward 比较 use/源号及 M/W valid/fault/RW/readiness	最近 M 匹配优先；M load 未就绪阻断旧 W	u_forward 内比较/sel/blocked 属控制、16 位选择属 DP；core 再选 ALU 输入

职责/对象	信息或意图来源	仲裁与资格	实际驱动与边沿
Z 与分支资格	core 选择 M CMP 候选、W 候选、已保存 Z；u_target 计算目标	EX 产生 branch_req，再经年龄与故障资格得到 qualified_redirect	pc16 采样重定向；目标计算不决定是否跳转，CMP 候选不直接写 Z
MEM 访问与故障	M 控制、完整 EA、valid/fault、rst 和 mode_active	m_address_in_range/m_access_fault；老 MEM 故障取消年轻 EX/ID/IF	core 驱动 dm_en/dm_we；同步 DM 在 MEM 结束边沿读或写，STR 只写一次
WB 架构写入	W 控制、valid/fault、rst 和 mode_active；同步 DM 给出 load 数据	core 形成 wb_ok、rf_we、z_we、cv_we	u_rf 与 u_flags 各自唯一更新架构状态；W 级 store/branch 标签不产生 RF 写入
流水数据与 CW	各级数据和意图形成下一载荷	core 唯一的占用仲裁计算 valid_next 及 fault_next	core 主时序块更新 D/X/M/W 载荷；有效性掩码保证同一条指令的数据与控制对齐
原因与停止现场	ID/EX/MEM 识别 reason；WB 停止 token 与 pipeline_empty_next 决定 drain_done	诊断使用同一有效标签；取消的年轻指令不保留现场	core 另一个时序块更新 x/m/w_reason 与 stop/last_completed，非全部流水字段同一 always
系统观察状态	cpu_system 采样合格 RF/MEM 写入事件	不参与任何执行选择或仲裁	system 时序块保存 RF 影子和最后 store；与架构 RF 不是两份执行资源

d_/x_/m_/w_分别绑定 ID/EX/MEM/WB。d_ctrl 及 CW_X/M/W 是表 22、表 23、表 15 中既有控制字段的图示集合，不是新增总线或寄存器；上层控制带与下层数据共用四个流水边界。图 1 的 FB 为反馈组，UPD 为 valid_next/取消/保持组。操作数前递由 operand_forward 完成，WB→ID 旁路位于 regfile4x16，Z 来源选择、load-use 检测、年龄仲裁和 mode 更新均位于 core 内部。M 寄存 m_fault 用于前递，新 m_access_fault 只进入访问保护、故障载荷及年龄取消，不接回前递数据 MUX。

3.1.3 本拍控制与时钟边沿

周期内先译码与判断资格，在结束上升沿更新相应流水载荷及架构状态。D 级译码只形成意图；RW/ZW/CVW 在有效、无故障的 W 槽才获准写 RF 或标志，MR/MW 在完整地址合法的 M 槽才获准访问 DM。mode 的 RUN/DRAIN 允许合格老指令继续，最终停止态关闭新访问与写入。具体使能表达式与年龄优先级见第 3.6 节。

PC 由唯一的 pc16 状态寄存器保存。JUMP 目标为 ZExt(imm8)，JZ 目标为 low16(x_pc+1+SExt(imm8))，二者在 EX 由 branch_target16 生成；WB 不再增加 PC。完整 CU 的输出还依赖当前输入，不能由 halted/fault 的状态型输出推断整个 CU 都是纯 Moore 逻辑。

3.2 随行意图 载荷与当拍动作

ID-CU 是指令语义转成后续操作意图的入口，但并不独自处理全部冒险和停止。各级通过同一套跨级仲裁接收推进、保持或取消结果，再结合本级意图与资格产生实际选择和写入。

CW_X、CW_M 和 CW_W 分别指既有 x_*、m_*、w_*控制字段的逻辑集合，总宽度为 18、7、5 位。它们不是新端口、独立控制器或额外寄存器；字段在最后使用后可以不继续传递。imm_signed 在 ID 已用于扩展，因此没有 x_imm_signed 寄存器。

表 21 三类信息的边界

类别	实际字段或信号	含义及约束
CW 操作意图	alu_op、a/b 选择、use_rd/rs、RW/MR/MW/ZW/CVW、wb_sel、jump/halt	描述这条指令后续需要什么；不等于当拍已经执行写入
数据与元数据	操作数/立即数/结果/store 数据；pc、rd/rs、valid、fault、reason、w_store	说明数据属于谁、是否有效和为何停止；候选 flags 是数据，valid/fault 是资格标签
当拍动作与下一状态	fetch_accept、pc_hold、qualified_redirect、dm_en/we、rf_we、z_we、cv_we、*_valid_next	由当前状态、CW 和相关事件计算；仅在规定边沿更新各自状态或存储器

3.2.1 ID 译码与指令操作意图

表 22 定义操作数路径。0/Rd/Rs/Imm 分别表示常数 0、已转发 Rd、已转发 Rs 和扩展立即数；imm_signed 对应 ID 扩展器 sign_mode。未使用选择项仍给安全的确定值。

表 22 指令译码操作数控制

Opcode	use_rd	use_rs	ALU a	ALU b	alu_op	imm_signed
0 MOV 寄存器	0	1	0	Rs	0	0
1 MOV 立即数	0	0	0	Imm	0	0
2 AND	1	1	Rd	Rs	3	0
3 OR	1	1	Rd	Rs	4	0

Opcode	use_rd	use_rs	ALU a	ALU b	alu_op	imm_signed
4 ADD 寄存器	1	1	Rd	Rs	1	0
5 ADD 立即数	1	0	Rd	Imm	1	0
6 SUB 寄存器	1	1	Rd	Rs	2	0
7 SUB 立即数	1	0	Rd	Imm	2	0
8 LDR	0	1	Rs	Imm	1	1
9 STR	1	1	Rs	Imm	1	1
A CMP	1	1	Rd	Rs	0	0
B 保留	0	0	0	0	0	0
C JUMP	0	0	0	0	0	0
D JZ	0	0	0	0	0	1
E HALT	0	0	0	0	0	0
F 保留	0	0	0	0	0	0

CMP 用合法 PASS_B 默认 op，但不使用 result；eq 比较选入的 Rd 与 Rs。JUMP/JZ 目标由 branch_target16 单独计算，输入为本条 pc 和 imm8，不能使用上表已置 0 的 ALU 结果。

表 23 定义写入及控制意图。RW/MR/MW/ZW/CVW 分别是 reg_write/mem_read/mem_write/z_write/cv_write；最终使能还需所属级 valid、故障和复位资格。

表 23 指令译码写入与控制掩码

Opcode	RW	MR	MW	ZW	CVW	其他控制
0 1	1	0	0	0	0	wb_sel=0
8	1	1	0	0	0	wb_sel=1
9	0	0	1	0	0	wb_sel=0
4 5 6 7	1	0	0	0	1	wb_sel=0
2 3	1	0	0	0	0	wb_sel=0
A	0	0	0	1	0	wb_sel=0
B F	0	0	0	0	0	illegal=1
C	0	0	0	0	0	jump_abs=1
D	0	0	0	0	0	jump_z=1
E	0	0	0	0	0	halt=1

各行未列出的 jump_abs/jump_z/halt/illegal 均为 0。组合译码先给默认值再 case 覆盖，不用 casex 掩盖未知 Opcode。CPU 保留 Opcode 与 ALU 非法 op 是两种错误来源，均须按有效指令检测。

Group 3 的 MemEn=1 只表示访存意图：LDR 的 MR/RW 为 1、MW 为 0；STR 的 MW 为 1、MR/RW 为 0。物理 dm_en/dm_we 须在 MEM 结合完整地址与有效资格产生。Group 7 的 1?11 只匹配 1011/1111，即 B/F；不能只检测低两位 11，否则会把合法 OR 和 SUB 立即数误判为非法指令。

3.2.2 后级将意图变为实际动作

例如 ID 译码得到 reg_write=1，只表示将来需要写 RF。该意图随指令推进，到 WB 后才由 WBCU 结合 mode_active、w_valid、w_fault 和 rst 形成 rf_we。错误路径的意图即使曾经为 1，被取消后也不能产生写入。

默认 AC1 的实际写入资格如下。STR 本身的 CW 将 RF/flags 写意图置 0，MEM 写入后仍保留有效 WB 完成标签；无副作用的有效阶段与 valid=0 的 bubble 不同。

```
rf_we = !rst && mode_active && w_valid && !w_fault && w_reg_write;
z_we  = !rst && mode_active && w_valid && !w_fault && w_z_write;
cv_we = !rst && mode_active && w_valid && !w_fault && w_cv_write;
dm_we = !rst && mode_active && m_valid && !m_fault
     && m_address_in_range && m_mem_write;
```

3.3 前递与流水协调的组成

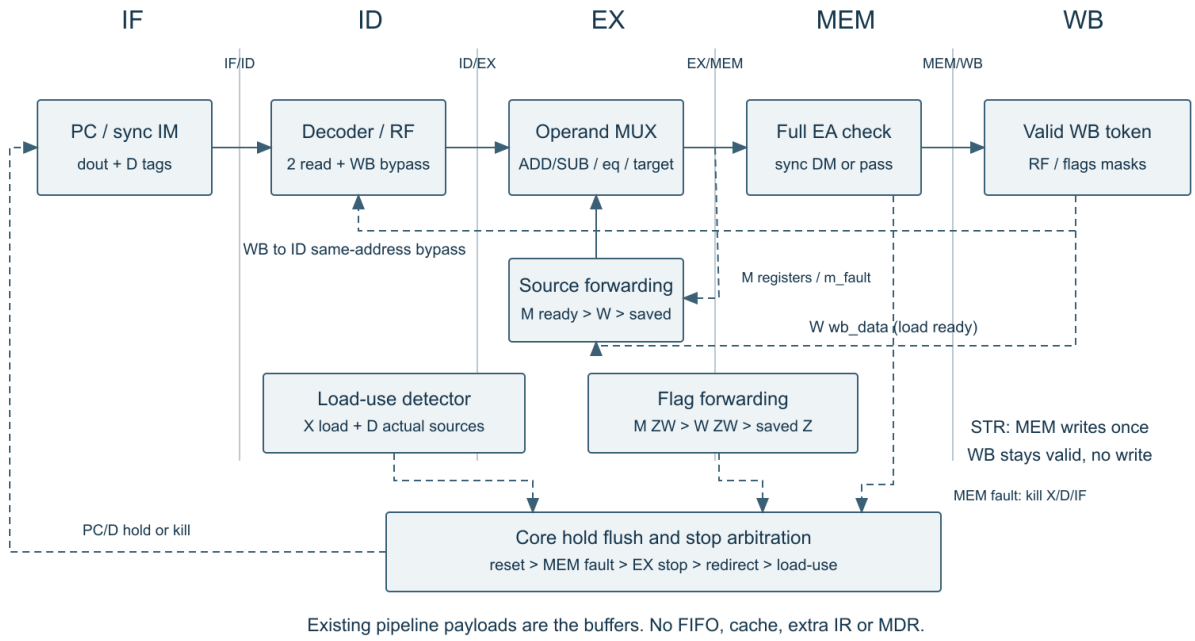


图 7 AC1 流水 Buffer 与 Interlock。M 寄存数据、valid/RW/就绪与 m_fault 用于源选择；完整 EA 检查用于 DM 保护及 MEM fault 取消，独立于前递数据 MUX。停顿、Flag forwarding 和年轻取消不变，不增加 FIFO/cache。

表 24 八项机制的实体与信号归属

机制	实现位置	主要接口与动作
HDU	core 内逻辑；未新增实体	X load/rd2 与 D 源/use/valid/fault；load_use1
操作数前递	operand_forward 原样复用	M/W 目标及资格；每路 sel2、fwd16、blocked1
WB 到 ID 旁路	RF 原样复用	rf_we、w_rd2、wb_data16，同址组合返回
Load-use interlock	core 内逻辑复用	PC/D hold，IM en=0，X bubble，M/W 推进
Flag forwarding	core 内 Z 选择复用	M/W ZW 候选及保存 Z；z_effective1
Redirect / Flush	core 仲裁复用	target16；取消 D/IF，MEM fault 另取消 X
Buffer / valid	现有流水载荷，core 最小修改	w_valid_next=m_valid；w_store1；无 FIFO/额外 IR/MDR
Stop / Reset	原 RUN/DRAIN/HALTED/FAULTED	停止 token+valid_next；按年龄仲裁，rst 最高

3.4 数据相关与 load use 停顿

每个实际源优先取最近的 M 级 RF 生产者，再取 W 级写回值，最后取 ID/EX 保存值。只有 M 级非 load 结果已就绪；M 级 load 的 exec_value 是地址，禁止把它当数据转发，也禁止越过未就绪的最近 load 使用更老同名值。

```
load_use = x_valid && !x_fault && x_mem_read && d_valid && !d_fault
       && ((d_use_rd && d_rd == x_rd) || (d_use_rs && d_rs == x_rd))
```

条件满足且没有更高优先级事件时，PC 和 IF/ID 保持，im_en=0，ID/EX 插入 bubble；当前 EX 的 load 继续推进，老 MEM/WB 照常完成。R0 参加相关比较。

表 25 load use 停顿与转发

周期	LDR R0,[R1]	ADD R2,R0	关键动作
C1	IF	未进入	正常
C2	ID	IF	正常
C3	EX	ID 保持	检测 load use；阻止覆盖 IF/ID；下周期 EX 插 bubble
C4	MEM	ID	DM 在结束边沿返回数据 消费者进入 EX
C5	WB	EX	从 W 的 load_data 转发 a/b 对应实际源
C6	已完成	MEM	普通结果传递
C7	已完成	WB	写 R2 及本条 C/V

LDR 到 CMP、STR 基址或 STR 数据的紧邻相关同样停 1 拍。LDR 后紧邻 MOV 同目的立即数不读取旧目的，无相关停顿。WB 到 ID 同址旁路同时保留，处理 ID 采样与 WB 写回在同拍的情形。

3.5 冒险分类与解决矩阵

下表完整列出当前 ISA 的实际冲突类别，精确展开的 2,376 个 RAW 实例位于 CPU 核心设计/说明/Hazard_Matrix_2376.csv。每例包含生产者/消费者、源角色、寄存器、g 间距、检测级、就绪级、选择机制、停顿数、周期模板、预期及实际结果。g 表示中间独立指令数。

表 26 当前 ISA 冒险类别与真实用例

类别	解决机制	停顿或验证实例
ALU/MOV 到 ALU、MOV 寄存器或 CMP	M 非 load 优先；中隔 1 条用 W，中隔 2 条 WB 到 ID	0；RAW_P0 至 7_C0/2/3/4/5/6/7/a_R*_S*_G*

类别	解决机制	停顿或验证实例
ALU/MOV 到 LDR/STR 基址	Rs 前递后接 ALU a，符号位移接 b	0；RAW_P0 至 7_C8/9_R*_S1_G*
ALU/MOV 到 STR 数据	Rd 前递独立保存 store_data	0；RAW_P0 至 7_C9_R*_S0_G*
LDR 到 MOV reg/ALU/CMP	相邻 load-use；用 W load 数据，禁止 M 地址	相邻 1，其余 0；RAW_P8_C0/2/3/4/5/6/7/a
LDR 到 LDR/STR 基址	同一 load-use 规则应用真实 Rs	相邻 1，其余 0；RAW_P8_C8/9_S1
LDR 到 STR 数据	真实 Rd 参与检测，W 前递到 store_data	相邻 1，其余 0；RAW_P8_C9_S0
Rd 与 Rs 相同	两路独立选择同一最新值；R0 亦同	0 或 1；RAW_*_S2 覆盖 648 例
多生产者同名	M 匹配先于 W；M load 未就绪阻止穿越旧 W	NEWEST 与 NEW_LOAD_AFTER_TWO_WRITERS
WB 与 ID 同址	RF 内显式旁路，ID 捕获本拍 wb_data	0；全部 G2 例
假相关	只比较 use 掩码允许的源	0；FALSE_MOV 64 例、FALSE_LDR_DEST 4 例
CMP 到 JZ	M ZW 优先 W 再 saved；0 覆盖旧 1	0 数据停顿；FLAG 84 例与原连续 CMP
taken / not-taken	taken 清 D 和 IF，自身 M/W 有效；not-taken 继续	taken 2 槽；FLUSH_CLASS_0 至 f
MEM fault 与 EX 事件	老 MEM 故障优先于 redirect/HALT/load-use	T16、MEM_FAULT_OVER_REDIRECT、MEM_FAULT_WITH_LOAD_USE
HALT / EX fault	老 M/W 推进，年轻取消，WB 消费停止 token	T07、STR_THEN_HALT、LEGAL_STR_BEFORE_EX_FAULT
同步存储/结构	Harvard 分离；DM 每拍一个 M 请求；RF 2 读 1 写	0 结构停顿；MEM_BACK_TO_BACK、TRACE_LOAD_STR
WAW / WAR / C/V 读取	固定 WB 顺序、老 EX 消费先于年轻 WB；无 C/V 读指令	架构排除，无额外 scoreboard

以下间距速查限于正常推进且无额外停顿或取消改变时间关系：0 至 7 生产者在 EX 未就绪，g=0 用 M、g=1 用 W、g=2 经 WB→ID 保存值；LDR 在 MEM 未就绪，g=0 停 1 拍后用 W，g=1/2 不增加停顿。实际按消费者到 EX 时最近合格生产者所在级、就绪与 valid 选值；C/V 无读取 RAW，WAW/WAR 仍由顺序阶段排除。

合法 branch_req 与 load_use 不会同拍成立：它们都要求同一 X 载荷，但 C/D 和 8 的译码互斥；这一点由译码合约和运行断言检查。真正的交叉事件是老 MEM fault 与年轻 X load 导致的 load_use、branch 或 HALT；老故障优先。原始 load_use 计数可为 1，但故障同拍取消年轻 X，不能把它描述成正常完成的一拍等待。

3.6 写入资格与事件优先级

以下表达式对应当前 RTL。w_fault 是 WB 载荷自身故障，m_fault 是 MEM 载荷自身故障；取消通过清 valid 实现，不另设 killed 寄存器。mode_active 允许 RUN 和两种 DRAIN 状态的老流水继续完成，不允许最终停止状态产生写入。

```
mode_active = (mode == RUN || mode == DRAIN_HALT || mode == DRAIN_FAULT)
wb_ok = mode_active && w_valid && !w_fault
rf_we = !rst && wb_ok && w_reg_write
z_we  = !rst && wb_ok && w_z_write
cv_we = !rst && wb_ok && w_cv_write
mem_ok = !rst && mode_active && m_valid && !m_fault && m_address_in_range
dm_en = mem_ok && (m_mem_read || m_mem_write)
dm_we = mem_ok && m_mem_write
```

停止前端不能屏蔽较老 WB。wb_ok 表示 WB 槽资格；STR 的 DM 副作用发生于 MEM，但指令完成统一在 WB 消费有效标签。当前版本在非复位的合格 WB 每拍至多数一条完成，包含 STR、分支、CMP 与 HALT；故障 token 不计合法完成，不再把 MEM store 额外加到 WB 计数。

MEM 错误仍只针对有效访存指令，自身故障或 EA 越界触发 m_access_fault/m_fault_event；普通 ALU 值超出 DM 容量不报访存错。AC1 把 u_forward.m_fault 从 m_fault OR m_access_fault 改接寄存 m_fault：合法 M 非 Load RF 生产者不访问 DM，M Load 仍 not-ready 并阻断旧 W。新 MEM fault 仍禁止 DM、取消年轻 X/D/IF 并保留老 WB，故不消费年轻组合结果；redirect 优先级不变。

表 27 流水事件的年龄与优先级

优先级	事件	前端与年轻指令	较老级
1	同步复位	PC/valid/停止状态清 0	RF 和 Z/C/V 清 0 复位边沿禁止 DM 写
2	MEM 自身故障	停取指 取消当前 EX 及更年轻动作	故障 MEM 不访问 老 WB 仍完成
3	EX 故障或 HALT	停取指 取消两个年轻槽	老 MEM/WB 继续 完成后停止
4	EX 合格重定向	PC 取目标 清年轻槽并取消错误路径 IF	老 MEM/WB 完成
5	load use	PC/IF/ID 保持 ID/EX 插 bubble	load 及老 MEM/WB 推进

优先级	事件	前端与年轻指令	较老级
6	正常推进	逐拍接收载荷	逐拍完成

保留 Opcode 在 ID 标记故障，EX 确认；它可能被更老分支取消，不能一译码就不可逆停机。MEM 越界比 EX 分支更老，故障优先于该分支 redirect。valid=0 时禁止 RF/DM/标志写入、redirect、HALT 和故障激活。

3.7 HALT 故障与复位

合格 HALT 要求处于 RUN、未复位、X 有效且没有自身故障，同时不存在更老 MEM 故障。EX 自身故障优先于本条 HALT 意图。HALT 在 EX 结束边沿使 mode 进入 DRAIN_HALT，随后 stop_pending 为 1；本边沿禁止新的取指请求，并取消年轻槽及对应 IF 响应。HALT 标记保留 valid，依次通过 MEM 和 WB，在 WB 结束边沿被消费。该边沿确认更新后的流水线为空后进入 HALTED，halted 置 1。HALT 前的 load 仍须写回，HALT 后的 STR 不得进入合格 MEM；不能仅因较老指令已经完成，就在 HALT 的 MEM 周期提前停止。

故障使用同样的 WB 结束规则。MEM 检测到越界时，立即阻断故障自身的 DM 访问，取消年轻 EX，把故障标记随 valid 送到 WB；本拍老 WB 仍可写 RF 或标志。故障 STR 保留 WB 的 valid 和 fault；成功 STR 同样保留有效 WB，只是无故障且不再产生写入。EX 故障只取消自身副作用和年轻指令，故障标记经 MEM 送到 WB，老 MEM 合法 STR 仍可写 DM。故障标记不允许写 RF、DM 或 Z/C/V；在 WB 消费标记并确认下一拍为空后进入 FAULTED，fault 置 1 直到复位。故障原因和 fault_pc 属于可选诊断，不实现异常返回或重试。

同步 rst 跨越有效上升沿，清 RF、Z/C/V、PC、全部 valid 及停止状态。每级复位优先于保持和推进。复位前已经完成的 DM 写入保留；复位当拍 dm_we 为 0。外部按钮同步与去抖在板级顶层完成，核心不用门控组合时钟。

3.8 全局运行模式 FSM

3.8.1 运行模式与事件定义

模式 FSM 只展开控制状态 Q 中的 mode 字段。其五个合法编码为 RUN=000、DRAIN_HALT=001、DRAIN_FAULT=010、HALTED=011、FAULTED=100。正常推进、load-use 保持和分支取消均可发生在 RUN 内；它们不构成全局串行的取指至写回状态循环。

输入事件采用已获资格的内部判定：E=m_fault_event||x_fault_event，H=qualified_halt，D=drain_done。E 包含有效 MEM/EX 故障；H 已经排除复位、自身故障及更老 MEM 故障；D 表示 WB 有效停止标记在本边沿被消费后，所有 valid_next 均为 0。rst 为同步高有效复位，优先于其他转移。

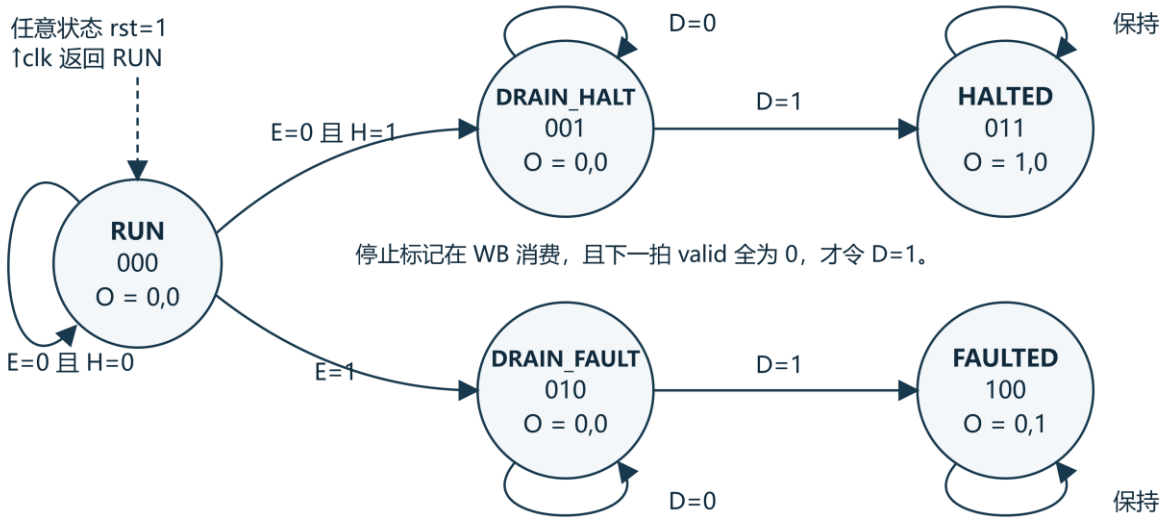


图 8 运行模式状态转移图。圆内 O=(halted,fault)为当前状态输出；箭头标转移条件。所有状态在 clk 上升沿更新，rst 优先。

3.8.2 状态转移与当前输出

halted=(mode==HALTED)，fault=(mode==FAULTED)，这两个输出满足 Moore 定义。表 28 输出列为边沿前的当前值；进入目标状态后再按新 mode 解释输出。RUN 不保证 im_en 恒为 1，取指仍受互锁、事件与地址资格控制。

表 28 CU 运行模式的完整转移与输出。除复位行外均取 rst=0。

当前 mode	输入条件	下一 mode	当前 O	边沿动作与结果
任意状态	rst=1	RUN	依当前态	清 PC/RF/标志/valid，禁本拍写入；已写 DM 保留
RUN 000	E=1	DRAIN_FAULT	0,0	禁止故障副作用，按年龄取消年轻槽
RUN 000	E=0,H=1	DRAIN_HALT	0,0	停取指；老合格指令与停止标记继续
RUN 000	E=0,H=0	RUN	0,0	正常推进、互锁或分支取消
DRAIN_HALT 001	D=0	DRAIN_HALT	0,0	老合格指令与 HALT 标记继续
DRAIN_HALT 001	D=1	HALTED	0,0	消费 WB 标记；边沿后 O=1,0

当前 mode	输入条件	下一 mode	当前 O	边沿动作与结果
DRAIN_FAULT 010	D=0	DRAIN_FAULT	0,0	老合格指令与故障标记继续
DRAIN_FAULT 010	D=1	FAULTED	0,0	消费 WB 标记；边沿后 O=0,1
HALTED 011	任意 E/H/D	HALTED	1,0	保持停止，禁止访问与写入
FAULTED 100	任意 E/H/D	FAULTED	0,1	保持停止，禁止访问与写入
101/110/111	任意 E/H/D	FAULTED	0,0	非法模式禁访问/写入；本沿转故障停止

3.8.3 排空完成与复位返回

HALT/故障被接受时，E/H 事件只启动排空，halted/fault 仍为 0。有效停止标记到 WB 且 D=1 的结束边沿，才分别进入 HALTED 或 FAULTED。停止态在 rst=0 时保持，只有同步复位返回 RUN；当前接口没有 start/resume。若把当前输出重复写在状态转移边上，DRAIN 到停止态的边仍标 0,0，边沿后才变为 1,0 或 0,1。

T07 的实际记录：C4 末接受 EX 中的 HALT 并取消年轻 STR；C5 末较老 LDR 仍可写 R1=1234；C6 末在 WB 消费 HALT 后才 halted=1。T16 中较老 LDR 的 EA=0100，C4 即阻断 DM 访问并取消年轻 HALT，C5 末消费故障标记后 fault=1。两例独立起算周期；较老 WB 可写回指的是 MEM 故障检测当拍，T05 另提供该同拍见证。

本基线的故障均在 EX 或 MEM 当拍确认，没有延迟错误响应。较老 MEM 故障与 EX 的 HALT 同拍时，RUN 直接进入 DRAIN_FAULT，HALT 被取消；一旦 HALT 被接受并进入 DRAIN_HALT，就不会再出现新的较老故障，因此不设置 DRAIN_HALT 到 DRAIN_FAULT 的转移。若后续引入可变延迟或延迟错误响应，必须先修订存储接口、年龄仲裁和状态图。

mode 为 3 位状态量；逻辑状态编码在 RTL 中统一声明。stop_pending 等于当前处于任一 DRAIN 状态。以下是组合判定关系，valid_next 来自本拍按复位、取消、保持和推进规则计算的下一状态。停止请求被接受时取指已关闭，未被接受的 IF 响应已作废。

```
w_stop_token = w_valid && (w_halt || w_fault);
pipeline_empty_next = !(d_valid_next || x_valid_next ||
                        m_valid_next || w_valid_next);
drain_done = !rst && stop_pending && w_stop_token && pipeline_empty_next;
```

WB 停止标记在本边沿消费，w_valid_next 清 0；若其余 valid_next 也为 0，同一边沿将 mode 更新为 HALTED 或 FAULTED，无需额外等待一拍。同步复位优先于此判定。没有有效 WB 停止标记时，即使当前没有 RF/DM 写入，也不得自行进入最终停止状态。

非法 mode 的 RTL 默认分支进入 FAULTED，不得自行回到 RUN 重新取指；mode_active 同时阻断非法状态下的访问及写入。原九套回归检查可达状态、停止边沿和复位。本版系统另以受控仿真注入非法二进制 mode，检查原因 6 及无指令 PC 的现场；不表示一般硬件容错。状态寄存器用非阻塞赋值，next-state 及输出组合逻辑完整覆盖。

3.9 五个实例贯通数据 CW 与当拍动作

以下 C1 至 C5 表示一条无额外停顿指令依次占据 IF、ID、EX、MEM、WB 的周期。每周期内组合计算，结束上升沿捕获结果。CPU 中的其他指令同时占据不同级，不能用一个全局 IF 至 WB 循环状态机每次只执行一条指令。

正常 IF 每拍推进 PC_fetch；EX 合格分支改为目标；load use 或停止保持前端。各指令的 pc 标签始终保留自身地址。RF 和标志的写回不负责再增加 PC，STR 完成也不负责更新 PC。

以下表格使用当前默认源码新生成的独立 48 例 CSV。阶段列以有效标签和 pc 读取；IF 列为实际 im_en 请求地址。写入值由 TB 逐边沿与独立 ISA 预期比较，控制意图解释据 ISA 和 RTL，不冒充未记录的 CW 总线波形。列头表示结束边沿之前所在阶段，最后一列说明该边沿动作。

3.9.1 相邻 ADD 的 M 到 EX 前递

初始 RF：R0=0001, R1=0002, R2=0004, R3=0000；初始 Z/C/V=000。这是 TB 列明的用例初值，不是新增 ISA 指令。

```
00: ADD R0,R1 (4100)
01: ADD R2,R0 (4800)
02: HALT (E000)
```

表 29 DOC_ADD_ADD 实际逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:ADD	—	—	—	正常推进
C3	02	01:ADD	00:ADD	—	—	正常推进
C4	03	02:HALT	01:ADD	00:ADD	—	正常推进
C5	—	03:RES	02:HALT	01:ADD	00:ADD	RF R0←0003；WB 写 C/V
C6	—	—	—	02:HALT	01:ADD	RF R2←0007；WB 写 C/V
C7	—	—	—	—	02:HALT	停止 HALTED

C4 第一条 ADD 在 M、第二条 ADD 在 X，最近 M 非 load 结果可直接前递；不等待 RF 在 C5 写入，也不增加停顿。最终 R0=0003、R2=0007。

3.9.2 默认 AC1 的 load use 保持与 bubble

初始 RF：R0=0000, R1=0021, R2=0000, R3=0020；初始 Z/C/V=000。这是 TB 列明的用例初值，不是新增 ISA 指令。

DM[0020]=0007；STR 的独立基址为 R1=0021。

```
00: LDR R0,[R3+0] (8300)
01: STR R0,[R1+0] (9100)
02: HALT (E000)
```

表 30 DOC_LOAD_STR 实际逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:LDR	—	—	—	正常推进
C3	—	01:STR	00:LDR	—	—	保持 IF/ID；X 插 bubble
C4	02	01:STR	—	00:LDR	—	正常推进
C5	03	02:HALT	01:STR	—	00:LDR	RF R0←0007
C6	—	03:RES	02:HALT	01:STR	—	DM[0021]←0007
C7	—	—	—	02:HALT	01:STR	正常推进
C8	—	—	—	—	02:HALT	停止 HALTED

C3 仅保持 IF/ID 并给下一拍 X 插 bubble，LDR 继续进 M；C5 的 STR 从 W 取得 load 数据，C6 写 DM，C7 仍保留 STR 的 WB 标签。默认 AC1 共 8 拍；候选的 7 拍属于独立新契约。

3.9.3 CMP 到 JZ 的 Z 前递与年轻取消

初始 RF：R0=0001, R1=0001, R2=0000, R3=0020；初始 Z/C/V=000。这是 TB 列明的用例初值，不是新增 ISA 指令。

```
00: CMP R0,R1 (A100)
01: JZ #+1 (D001)
02: STR R0,[R3+0] (9300)
03: HALT (E000)
```

表 31 DOC_CMP_JZ 实际逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:CMP	—	—	—	正常推进
C3	02	01:JZ	00:CMP	—	—	正常推进
C4	—	02:STR	01:JZ	00:CMP	—	EX 重定向；取消年轻槽
C5	03	—	—	01:JZ	00:CMP	WB 写 Z
C6	04	03:HALT	—	—	01:JZ	正常推进
C7	—	04:RES	03:HALT	—	—	正常推进
C8	—	—	—	03:HALT	—	正常推进
C9	—	—	—	—	03:HALT	停止 HALTED

JZ 在 X 使用 M 中最近 CMP 的候选 Z=1，无需等待架构 Z 写回。pc2 的 STR 被取消，不得写 DM；CMP 仍在 WB 更新 Z。目标 pc3 的 HALT 按完整路径停止。

3.9.4 STR 在 MEM 写入并在 HALT 前完成 WB 标签

初始 RF：R0=BEEF, R1=0021, R2=1234, R3=0000；初始 Z/C/V=000。这是 TB 列明的用例初值，不是新增 ISA 指令。

```
00: STR R0,[R1+0] (9100)
01: HALT (E000)
02: STR R2,[R1+0] (9900)
```

表 32 DOC_STR_HALT 实际逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:STR	—	—	—	正常推进
C3	02	01:HALT	00:STR	—	—	正常推进
C4	—	02:STR	01:HALT	00:STR	—	DM[0021]←BEEF
C5	—	—	—	01:HALT	00:STR	正常推进
C6	—	—	—	—	01:HALT	停止 HALTED

C4 老 STR 写 DM 且年轻 HALT 在 X 接受停止，老指令不能被 HALT 取消；STR 在 C5 继续有效 WB 但不再写 DM，HALT 在 C6 的 WB 结束边沿使核心停止。

3.9.5 较老 MEM 故障胜过年轻 EX 分支

初始 RF：R0=0000, R1=0000, R2=0000, R3=0000；初始 Z/C/V=000。这是 TB 列明的用例初值，不是新增 ISA 指令。

```
00: MOV R1,#255 (14FF)
01: ADD R0,#1 (5001)
02: LDR R2,[R1+1] (8901)
03: JUMP #5 (C005)
04: STR R0,[R0+0] (9000)
05: HALT (E000)
```

表 33 MEM_FAULT_OVER_BRANCH 实际逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:MOV	—	—	—	正常推进
C3	02	01:ADD	00:MOV	—	—	正常推进
C4	03	02:LDR	01:ADD	00:MOV	—	正常推进
C5	04	03:JUMP	02:LDR	01:ADD	00:MOV	RF R1←00FF
C6	—	04:STR	03:JUMP	02:LDR	01:ADD	RF R0←0001；WB 写 C/V；M 越界压制 EX 跳转；老 WB 保留
C7	—	—	—	—	02:LDR	停止 FAULTED

C6 的 M 级 LDR 完整 EA=0100，访问被禁止，年轻 X 级 JUMP 不获重定向；更老 W 级 ADD 仍写 R0 与 C/V。故障标签继续 W，在 C7 消费后 FAULTED。故障不会回滚已经完成的更老写入。

4 系统集成与验证结果

以完整系统层次组织实现、功能及性能证据。历史、有限输入、当前新运行与尚未完成的实物验收分别说明。

4.1 系统层次与启动观测

datapath_core 提供执行核心及同步存储端口；cpu_system 连接 IM/DM 并维护只读结果影子；ees331_top 通过 application、时钟缓冲与 VIO 形成课程板离线实现。三种顶层的资源统计边界不同，不能把核心、系统 OOC 和含 VIO 的板级资源当作同一对象比较。

observe_wb_valid 表示当前有可观察 WB 标签，仍可能是 fault、STR、branch 或 HALT；合法完成还要排除 wb_fault，RF 写入还要检查 observe_rf_we。这三个条件的计数口径不同。

cpu_system 将 13 份核心 RTL 与一个系统顶层连接。IM_INIT_FILE 和 DM_INIT_FILE 指向完整 256×16 字镜像；初始化仅在配置或仿真启动时发生。读请求一拍更新、禁用保持、DM 写周期保持 dout，reset 禁止当拍写入但保留已提交 DM。完整 EA 在核心检查后才截为物理 8 位地址。

追加接口分为合格 WB 事件、实际 stall/redirect、流水有效位、架构标志和稳定停止现场。WB 有效标签包含 STR/分支/HALT 及故障 token，不能等同 RF 写回或合法退休。系统由合格写回维护 RF 影子，由合格 MEM 写入保存最后 store；两者不反馈执行。停止原因和公开接口见第四章系统集成部分。

4.1.1 文件初始化与系统连接

阶段 2 以 cpu_system 为共同仿真和综合顶层；本版板级工程以 ees331_top 连接该系统。默认 all_ops 镜像覆盖 14 个合法 Opcode，错误路径的 store 和非法指令必须取消。系统 TB 仅通过文件启动，不层次写入 IM。

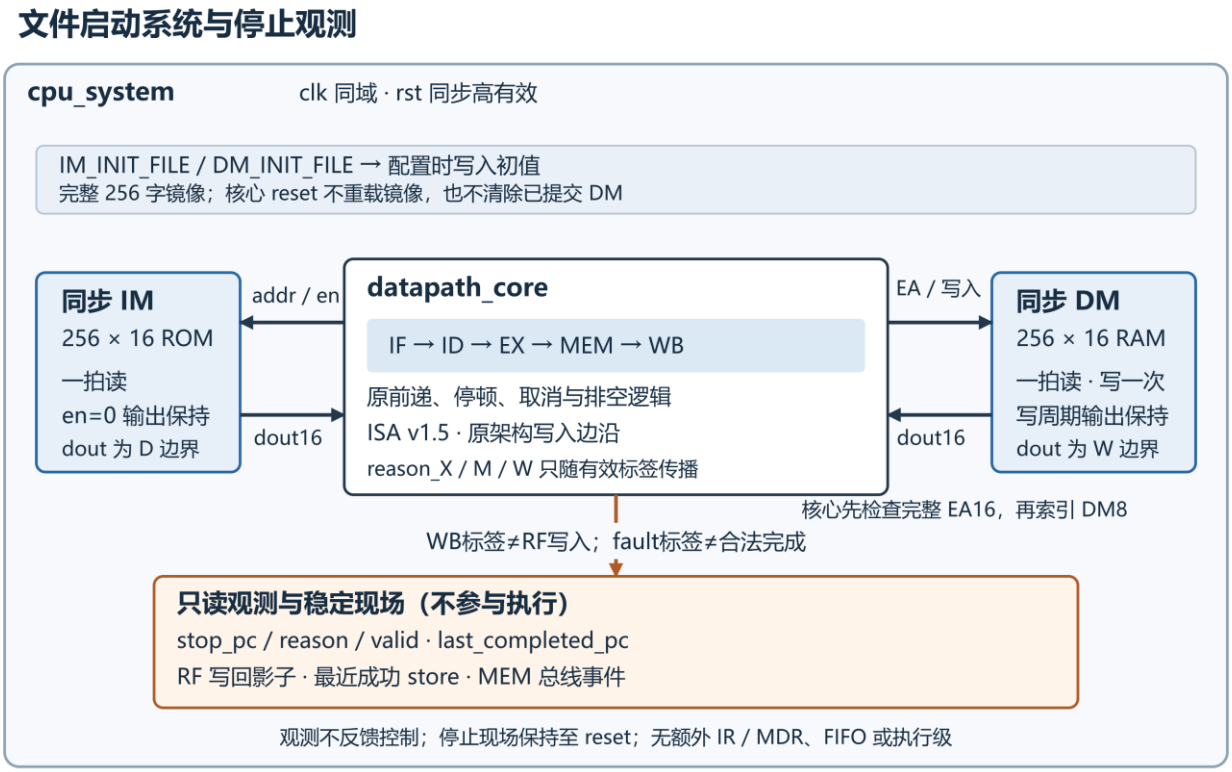


图 9 文件启动系统与停止观测。IM/DM 输出继续承担原 D/W 数据边界；框图不新增流水级。观测区域包含 core 的停止锁存，以及系统顶层中的 RF 写回影子、最后 store 记录。

INIT_FILE 只接受预先准备的 256 行四位十六进制文件。runner 在启动工具前检查文件清单、哈希、字数与格式；更换镜像后重新生成独立工程和新 attempt，不能沿用旧成功。系统同步高有效 rst 不是板上异步按键的直接接法，本版已在板级实现请求同步和释放去抖，见第 4.2 节。

4.1.2 稳定停止与公开结果接口

新增 107 位 core 观测输出不属于执行接口的 87 位；它们均无控制反馈。系统另暴露 RF 影子、MEM 事件和最后 store。载荷仅在各自 valid 或写使能有效时解释，不能把 bubble 中的旧数据当成事件。

表 34 系统公开观测接口

信号组	位宽与采样规则
WB 事件	valid/fault/halt/store 各 1；rd2、pc16、data16、候选 Z/C/V3；采样上升沿前的合格事件
写入与流水	rf/z/cv_we 各 1；stall/redirect 各 1；fetch_pc16、{D,X,M,W} valid4、架构{Z,C,V}3
稳定停止	stop_valid/stop_pc_valid 各 1、stop_pc16、stop_reason3；原 drain_done 边沿后保持至 reset

信号组	位宽与采样规则
完成标签	last_completed_valid1、last_completed_pc16；只记录无故障合格 WB，含 STR 和 HALT
系统结果	rf_snapshot64={R3,R2,R1,R0}；MEM en/we 各 1、addr/data 各 16；last_store_valid1、addr/data 各 16

表 35 停止原因编码

原因	含义	PC 规则
0	尚无停止现场	stop_valid=0
1	HALT	WB 中 HALT 自身 pc
2	非法 Opcode B/F	故障指令 pc
3	取指越界	越界取指标签 pc
4	数据 EA 越界	LDR/STR 指令 pc
5	内部控制非法或消费未就绪源	对应 EX 故障 pc
6	非法二进制 mode 的既有 fail-safe	stop_pc_valid=0，无指令 PC

reason 按 X→M→W 随有效标签传递；MEM 访存故障按年龄覆盖年轻 EX 事件。正常或指令故障停止在原 drain_done 边沿锁存 w_pc，不使用 PC_fetch；取消的年轻标签没有现场。reset 清 RF/flags、影子、流水有效位及现场，不清 DM。原因 5/6 只做受控仿真注入验证，不代表一般硬件容错。

4.1.3 实际结果与验收边界

表 36 文件启动系统与已存工具验证

检查	真实执行结果与范围
独立用例与回归	冻结 48 例、10 个 reset 见证、58 行结果；九套回归 19 个 native 步骤；五类 mutant 匹配预期失败；另有 X/Z、证据和生命周期检查
同步存储	7 项公开端口检查：文件读、逐拍更新、禁用保持、写时保持与写后读
系统程序	11 个程序、12 行完整 epoch 结果；全合法 Opcode、控制/访存故障、年轻取消、HALT 前老访存、两类 reset 与停止保持
周期与网表	baseline=17/10（周期/动态指令），1 停顿 1 跳转；all_ops=25/16，1 停顿 2 跳转；all_ops 网表回放完全同拍

检查	真实执行结果与范围
工程启动	实际创建 14 份 RTL 的 Vivado 工程，并经 launch_simulation 完成相同 11 程序检查
系统 OOC	Vivado 2025.2；xc7z020clg484-1；10 ns；376 LUT、431 FF、2 RAMB18、15 CARRY4；WNS=+1.363 ns、TNS=0.000、0 失败端点

阶段 1 至 2 的完整工具复跑：验证/tools_50040838e0ec474680085bb91e8fb001。系统仿真、综合、网表及工程检查：验证/cpu_system_n0r4buws。各 attempt 保存输入镜像、指纹、实际日志和 CSV；失败尝试保留 FAIL，不被后续成功覆盖。

阶段 2 系统 OOC 含双存储、观测状态和结果影子，不能与旧 core-only 资源直接混比；其记录未包含完整 IO 约束或布局布线。本版新增的板级布线和实际 setup/hold 检查另见第 4.2 节。ZPS7-1 对应的实际配置/启动仍待接板确认；没有全状态形式证明、全算术穷举、可变延迟存储或硬件单步的通过结论。

工具优先级为显式 CLI、tools.local.json、CPU_VIVADO_BIN/CPU_PYTHON、PATH；共享示例为 tools.example.json，个人路径不提交。仓库外当前目录也可用脚本绝对路径启动。系统工程以 create_project.tcl 生成至新目录；已有工程拒绝覆盖，镜像变化后重建。

4.2 板级时钟复位与观测边界

板级离线实现以 EES-331 手册的 xc7z020clg484-1、M19/100MHz 和 L18/S1 为目标。实际器件型号、实物板型与 JTAG 目标必须逐一对应；仅识别到 xc7z020 或显示 Programmed，不能证明课程 CPU 已通过实板验收。

实物验收已按用户要求延期。目标对应未确认的配置/诊断尝试均排除在本 CPU 的功能、时钟和性能证据之外；不能据此宣布 EES-331 时钟或 CPU 存在缺陷。

4.2.1 时钟复位与观测连接

ees331_top 以 BUFG 分发 M19 的 100MHz 晶振，VIO 与 CPU 应用逻辑共享时钟。S1 为 L18 低有效输入；相关 PL Bank 为 3.3V，LED 为高电平点亮。板级仅有时钟、按键和 8 个 LED 逻辑 IO。

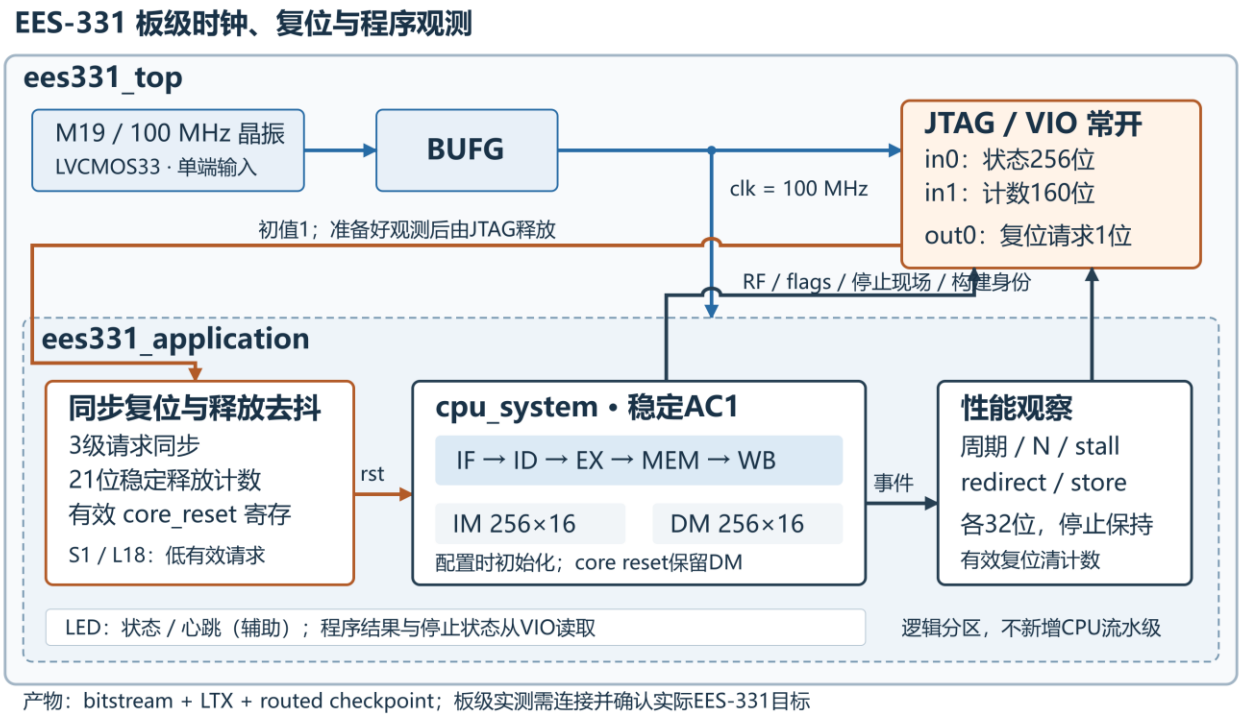


图 10 EES-331 板级结构。虚线内为同一 application 的逻辑分区；不新增 CPU 流水级。VIO 不受 CPU 复位，能够在停止时读取完整现场。

复位请求先经三级同步，再寄存为有效 core_reset；请求传递之前的合法写入可以完成，不能视作回滚。有效复位禁止当拍架构写入并清 CPU 和计数，DM 保留。请求撤销后保持 2^21 个稳定周期（约 20.97ms），再启动 CPU；CPU 执行时间不包含该等待。避免异步复位 FF 直接驱动 RAM 使能，原 REQP-1840 风险已在 RTL 中消除。

4.2.2 完整实现与时序覆盖

每次新建工程并显式保留 programs 路径；仿真与综合使用同一默认 dut_im.mem/dut_dm.mem。工具版本必须确为 2025.2，布局布线策略、逻辑 RAM 和 VIO 配置固定。检查 setup、hold、pulse、四项厂商 debug bus-skew、内部约束覆盖及十个实际 IO；只有按钮到首级同步 D 和无同步接收器的 LED 使用用户 false-path。

保留并按范围处理 PDCN-1569 和 LUTAR-1（厂商 debug hub）、RTSTAT-10（无负载）、ZPS7-1（仅 PL/JTAG，实际启动待板测）及 SYNTH-6（IM/DM 固定一拍读，不另加输出流水级）。未降级 DRC；新代码或影响 CPU 的异常范围不能自动接受。无矢量功耗估算未作为测量结果。

4.3 源码边界与连接约定

当前开发入口为 Git 工作副本 CPU 核心设计。Vivado/rtl 保留 13 个核心源码文件，Vivado/tb 保留 9 个 TB 入口和 2 份共享.svh；系统/rtl 另有 cpu_system.sv。core 仍集成原流水控制、前递、保护和停止，追

加公开观测；sync_memories 提供文件初始化同步存储。Pre5 原包仍是观测扩展前快照，不再声称其 13 份 RTL 与当前文件逐字节相同。

AC1 相对冻结 v1.6 的原组合优化及 72d99981、eae32d52 等记录保留为历史。阶段 0 至 2 仅在 datapath_core 追加观测输出、X/M/W 原因字段和停止锁存，在 sync_memories 追加 INIT_FILE 及映射属性；其余 11 个核心 RTL 与基线 7753502 相同。原 9 套 TB 不改；独立 48 例的私有运行副本仅把原.* 连接替换为 12 个执行端口的显式连接，断言与冻结镜像不变，并分别记录包内与执行后 TB 指纹。

当前维护入口为 Vivado/verify.py 与系统/run_system.py，使用统一工具配置；每次执行保留新的 attempt、源码镜像和原始日志。新失败不替换成旧 PASS。verify_results.py 的历史再校验不代表新运行 XSim；当前回归、系统仿真、综合和工程启动各有实际执行记录。

```
python -B -X utf8 .\CPU 核心设计\Vivado\verify.py --suite all
python -B -X utf8 .\CPU 核心设计\系统\run_system.py --test all
```

表 37 当前源码与连接约定

源文件或模块	与本规格关系	接入要求
rtl/regfile4x16.v	已有 4×16 RF 含 WB 到 ID 旁路	we 接合格 rf_we；两读口接 Rd/Rs
rtl/pc16.v	已有 reset > redirect > hold > increment	redirect/hold 由资格与优先级控制生成
rtl/imm_ext16.v	已有零/符号扩展	sign_mode 来自本条新译码
rtl/branch_target16.v	已有绝对/相对目标计算	branch_pc 来自 EX 随行标签
rtl/alu16.v、addsub16_cla.v	AC1 用 17 位求和推断进位链；C/V 接口不变	core→alu16→addsub；cla16/cla4 仅独立单元
rtl/isa_decode.v	已采用 AND/OR=2/3、LDR/STR=8/9	控制表与独立合约均已遍历 65,536 字
rtl/flags_reg.v	已实现独立 Z/CV 写入和同步复位	继续核对保存字段及指令写掩码
rtl/datapath_core.sv	原流水、仲裁、停止不变；M 前递接寄存 m_fault	MEM 故障年龄取消仍保留；AC1 有限回归已绑定
rtl/sync_memories.v	已实现固定单周期同步 IM/DM	保持 en 保持及标签对齐；DM 不随复位清空
九个 TB 入口及两份.svh	合约、单元、主回归、压力、手写事件检查	独立 ISA 预期与手算边界、真值表前递及逐拍检查；绑定当前源码和日志

下面源代码片段摘自现有 RF 和 PC，说明时序写入、读旁路和 PC 优先级。它们是单元代码，外围连接须满足第三部分的控制资格。

```
// regfile4x16.v  synchronous write and explicit WB-to-ID bypass
always @(posedge clk) begin
  if (rst) for (i=0; i<4; i=i+1) regs[i] <= 16'h0000;
  else if (we) regs[waddr] <= wdata;
end
always @* begin
  rdata_a = regs[raddr_a]; rdata_b = regs[raddr_b];
  if (we && !rst && waddr == raddr_a) rdata_a = wdata;
  if (we && !rst && waddr == raddr_b) rdata_b = wdata;
end
// pc16.v  redirect is already qualified by the control unit
always @(posedge clk) begin
  if (rst) pc <= 16'h0000;
  else if (redirect) pc <= target;
  else if (!hold) pc <= pc + 16'd1;
end
```

4.4 数据通路与控制的 RTL 连接

下列连接用于对照第二章接口，完整顶层见 AC1 datapath_core.sv。片段省略载荷和仲裁；alu16 接口不变，内部 u_addsub 为 17 位求和实现。原 flags_reg 与写资格沿用，全部有限回归已绑定 AC1 源码。

```
always @* begin
  alu_a = 16'h0000; alu_b = 16'h0000;
  case (x_alu_a_sel)
    2'd1: alu_a = rd_fwd;
    2'd2: alu_a = rs_fwd;
  endcase
  case (x_alu_b_sel)
    2'd1: alu_b = rs_fwd;
    2'd2: alu_b = x_imm_ext;
  endcase
end
alu16 u_alu(.a(alu_a), .b(alu_b), .op(x_alu_op),
  .result(alu_result), .eq(alu_eq), .illegal(alu_illegal),
  .carry(alu_c), .overflow(alu_v));
assign x_store_data = rd_fwd;
branch_target16 u_target(.branch_pc(x_pc), .imm(x_imm_ext[7:0]),
  .relative(x_jump_z), .target(branch_target));
assign wb_data = w_wb_sel ? dm_dout : w_exec_value;
regfile4x16 u_rf(.clk(clk), .rst(rst), .we(rf_we),
  .waddr(w_rd), .wdata(wb_data), .raddr_a(d_rd), .raddr_b(d_rs),
  .rdata_a(d_rd_value), .rdata_b(d_rs_value));
flags_reg u_flags(.clk(clk), .rst(rst), .z_we(z_we), .cv_we(cv_we),
  .z_in(w_cmp_z), .c_in(w_c), .v_in(w_v), .Z(Z), .C(C), .V(V));
assign dm_addr = m_exec_value;
assign dm_din = m_store_data;
```

wb_sel 随 load 标签进入 W；dm_dout 须遵守固定同步读契约并与 w_rd/w_valid 对齐。ALU 的 eq/C/V 和 ZW/CVW 由 EX/MEM、MEM/WB 保存后再接 flags_reg，不能直接把当前 EX 输出写入架构标志。操作数选择码 3 须由有效 EX 非法控制检测捕获；默认 0 赋值避免推断锁存器，不能取代错误检测。

流水 always 块使用非阻塞赋值；组合块覆盖全部情况并使用阻塞赋值。每个状态只由一个时序块驱动，load use 只保持前端，不用一个全局 enable 冻结全部级。

4.5 分层验证方法与证据范围

验证沿着指令语义、模块契约、流水逐拍行为和系统接口展开。独立 ISA 模型提供架构预期，TB 在每个有效边沿检查实际 RF/DM/flags、副作用顺序和停止状态；最终值一致不能替代前递、stall、redirect 或停止时刻的检查。

本轮文档重构未改变 14 份生产/系统 RTL 及两个隔离候选，未新增综合或布局布线执行。原九套回归和三组五程序实现按原身份复核；本轮另对当前默认源码真实重跑独立 48 例，并从该批实际 CSV 提取第三章的五个实例。

表 38 分层验证与当前证据状态

层次	权威证据	本轮状态与限度
源码与职责	14 份默认 RTL 字节等于 0d888cc；CW 字段和时序块所有权核对	无 RTL 变更；控制集合不是新增总线或模块
当前独立 ISA 检查	新 XSim：48 例、10 复位见证、58 结果、952 逐拍行	本轮真实重跑；五个正文例直接来自该 CSV
原九套回归	0d888cc 已存绑定源码的 9 套/19 原生步骤	沿用原始执行身份，不声称本轮重跑九套
文件启动系统	已存 11 程序/12 epoch、同步 RAM 契约、网表和工程启动	与当前 14 份源码一致；原结果范围保持
板级实现与候选	AC1 及两候选各五程序的已存完整布局布线	软件/硬件输入未改，不重新宣称执行 P&R
实板	用户延期；未确认目标的配置尝试已排除	没有 EES-331 功能或实测时间 PASS

4.5.1 自检查测试方法

TB 在边沿前采样使能，NBA 更新后检查状态。关键控制用$isunknown 检查 X/Z；停止与复位的禁止使能必须确定为 0，无效或未消费的数据载荷不强制已知。tb_units.sv 用独立精确整数 arithmetic_oracle 计算低 16 位、无借位 C 和补码范围 V，比较 result/C/V/illegal/eq，使用!==使未知输出失败。下面是内部 ALU 检查节选：

```
expected_c=0; expected_v=0; expected_bad=0;
case (f)
  0: expected_result=y;
  1: arithmetic_oracle(x,y,0,expected_result,expected_c,expected_v);
```

```
  2: arithmetic_oracle(x,y,1,expected_result,expected_c,expected_v);
  3: expected_result=x&y;
  4: expected_result=x|y;
  default: begin expected_result=0; expected_bad=1; end
endcase
#1;
if ({illegal,alu_c,alu_v,alu_result} !==
    {expected_bad,expected_c,expected_v,expected_result} || eq !== (x==y))
  $fatal(1, "ALU mismatch");
```

#1 是测试等待，不是硬件传播延迟。算术检查同时包含 C/V 预期：ADD 用至少 17 位精确和判断 C；SUB 直接用无符号 a≥b 判断无借位。V 的参考计算先按 16 位补码解释 a/b，再符号扩展到 32 位整数，计算精确结果并判断是否超出−32768 至 32767；不能直接把原无符号输入零扩展后用于 V 判断。先用第二部分手算向量检查参考模型，再做批量回归。

保留的 CLA4 单元仍检查 sum/cout、P/G 数值及 cin 独立性；独立 cla16 检查组间超前 K1 至 K4 和无组合环。这些结构检查不等于 AC1 运算路径检查。AC1 addsub/alu 按精确整数参考检查 result、无借位 C、V 与 eq，并按新源码指纹引用本次单元结果。

4.5.2 既有实现与验证证据

4.5.2.1 核心既有验证记录

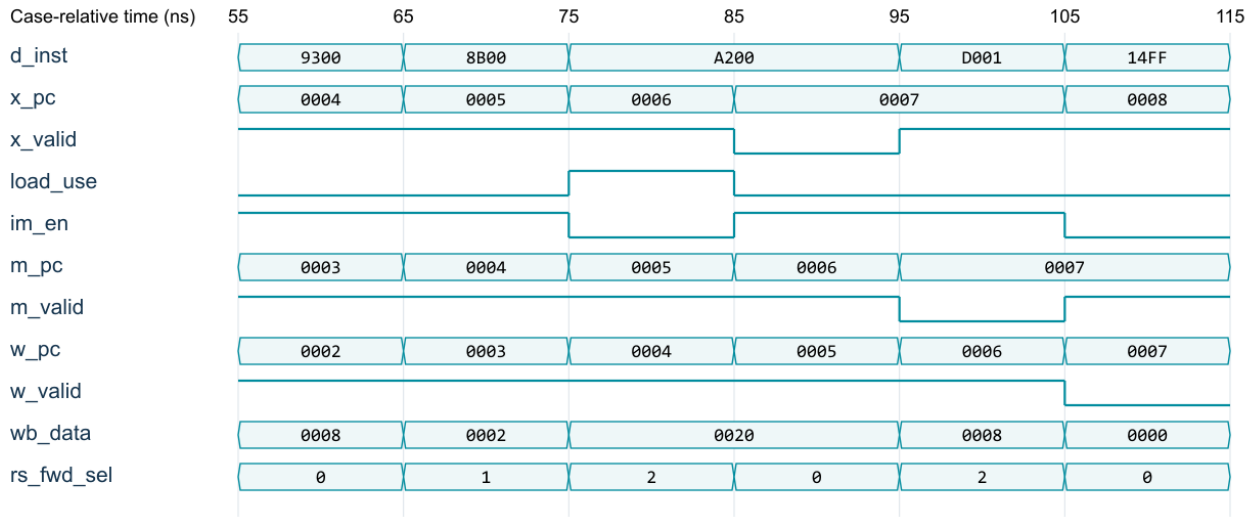


图 11 冻结 v1.6 历史基准 load-use VCD 窗口，保留原来源身份。AC1 对应逐拍 CSV 与冻结记录一致，可用于理解不变的周期契约；此图不是 AC1 新生成波形，也不是传播延迟测量。d_inst 为 ID 机器字，EX 按 x_pc/x_valid 定位。

表 39 v1.6 基准波形与逐拍记录的对应关系

周期位置	有效载荷或条件	观测结果	检查的行为
C9 边沿前	X 的 LDR pc6，D 的 CMP pc7	stall=1，im_en=0；M 的 STR pc5 有效	只保持前端，老 STR 在本边沿写 DM[0020]=0008

周期位置	有效载荷或条件	观测结果	检查的行为
C10 边沿前	M 的 LDR pc6，X 为 bubble	dm_en=1 dm_we=0；DM 读在结束边沿返回	load 与 WB 标签同边沿对齐；老 STR 为有效 WB，无架构写入
C11 边沿前	W 的 LDR pc6，X 的 CMP pc7	W 返回 0008 并转发至 CMP 实际源	不将 M 级 load 地址当作数据
C12 边沿前	X 的 JZ pc8，M 的 CMP pc7	redirect=1；目标 000A；候选 Z=1	最近 CMP 前递，取消年轻 pc9 及 IF 响应
C15 至 C17	HALT pcA 依次占 EX MEM WB	C17 结束边沿 halted=1，fault=0	WB 消费停止标记；10 条动态指令 17 周期

AC1 在 2026 年 10 月 14 日用 XSim2025.2 实际完成九套回归。合约检查 65,536 字和 14 项 ALU 边界；主回归 128 程序、16 次复位、3,380 次有效完成及 14 种合法 Opcode；stress 为 4,696 例、33,134 次完成；audit 为 12 定向加 4 重启，均零失败。基准仍 17 周期、10 动态指令、1 次 load-use、1 次取分支。

AC1 单位记录：独立 CLA4=512、独立 CLA16=564,300、当前 addsub=564,300、当前 ALU=2,257,200，译码 65,536、扩展 512、flags512、RF262,161、PC65,539、目标 3,584。CLA 单元 PASS 不表示其参与 AC1 CPU 路径。9 份结果/逐拍 CSV 与冻结 v1.6 的 r1 运行 75620c56 逐字节一致；仍非完整算术或 CPU 状态穷举。

迁移前的独立检查发现旧译码器有 16,384 字与新 ISA 不相容，差异位于 2/3/8/9。该诊断和原 Pre4 源码、日志作为历史记录保留；当前新译码器已通过另一次完整二进制指令字检查。合约 TB 另有 14 项 ALU 定向边界及非法选择检查通过，仍不代表完整 ALU 输入穷举。

完整 VCD 继续保留各级 pc/valid、操作数与转发选择、exec_value、store_data、dm_en/we/dout、WB 地址/数据/资格、候选与保存的 Z/C/V，以及 stall/redirect/kill。图 11 只选取基准中的 load 相关窗口；故障及其他指令按 case_timings.csv 定位到完整波形，并配合对应日志核验。

集成波形分别标出“输入已确定”“组合结果产生”“边沿锁存”“架构写入”，不要把总线有值当作有效指令完成。CU 检查增加 mode、branch_req 与 qualified_redirect、HALT/fault 请求和 drain_done，确认请求因年龄冲突被拒绝时没有副作用。波形截图须注明对应源码、TB 和程序镜像，配合零失败日志。

4.5.2.2 Pre5 控制验证记录

Pre5 工程于 2026 年 10 月 17 日完成同源九套回归，attempt 为 440e0d7a-1244-4333-a4f8-5adf612f9833，绑定 19 次原生工具调用；记录保存在课程目录 Pre5_控制单元设计/Vivado/验证记录下。

覆盖 65,536 个二进制指令字、128 个主回归程序与 16 次复位、2,550 个冒险案例，以及 131,072 个前递控制组合（4 个数据哨兵）。这些计数与核心既有覆盖口径一致。

同工程独立记录 cpu_ac1_independent_glok75my 包含 48 个案例、10 次复位后重启见证和 952 行逐拍记录。结果 CSV 的 58 行包括复位重启后的第二轮，案例数仍为 48。control_traces 保存 BASELINE、T01 至 T05、T07、T16、T17a、T18b 及 TRACE_LOAD_STR 共 11 份逐边沿记录；pre_为边沿前值，post_为同时间戳更新稳定后的值。

2026 年 10 月 29 日合入前，已按当前输入重新核验上述历史日志及指纹，重新解析两份原始 VCD 的 4,824 个上升沿，并核对全部 11 份 CSV/JSON 与关键 FSM、复位和写入资格。该步骤是历史证据复核，没有产生新的 XSim、综合或下板运行；现有功能通过范围仍是有限输入与程序集合。核心包和 Pre5 包的执行日期、入口及结果分别保留。

4.5.3 统一五级与冒险验证结果

表 40 AC1 已存实际运行范围

对象	实际检查数量	结果与边界
译码/边界	65,536 字及 14 手算 ALU 例	PASS；ISA v1.5 不变
原主回归	128 程序+16 复位，3,380 有效完成	PASS；128 最终状态和周期相同
原单元	独立 CLA16 及 AC1 addsub 各 564,300，ALU2,257,200	PASS；限定输入范围
原压力	4,696 案例，33,134 有效完成，1,408 fault tokens	PASS；加统一 WB 数量与 M/W 标签检查
原独立手写	12 定向+4 重启	PASS；旧架构状态预期保留
RAW 矩阵	1,728 单源+648 同源双读=2,376	PASS；9 生产者、10 消费者、4 寄存器、3 距离
假相关/Z/交叉	68 / 84 / 22；另复跑 16 复位	PASS；新 hazards 总计 2,550
前递控制空间	2^17=131,072；4 个不同数据哨兵	PASS；非完整 16 位数据穷举
统一 WB smoke	5 完成；1 DM 写；1 STR WB；1 branch WB	PASS；v1.5 同测试失败属历史
短时序样例	12 周期，5 完成，1stall，1redirect	PASS；原生 interlock.vcd 保留

主 TB 对 D/X、X/M、M/W 的有效位和 pc 推进、stall 时 PC/D/IM 保持、X bubble 及无效槽副作用逐边沿检查；统一 WB 完成模型独立于 MEM 架构写事件，防止成功 STR 丢失或重复统计。压力循环另逐边沿检查 M/W 标签与完成总数。相同生产者有多个候选时，未就绪 load 不得穿越取老同名值。

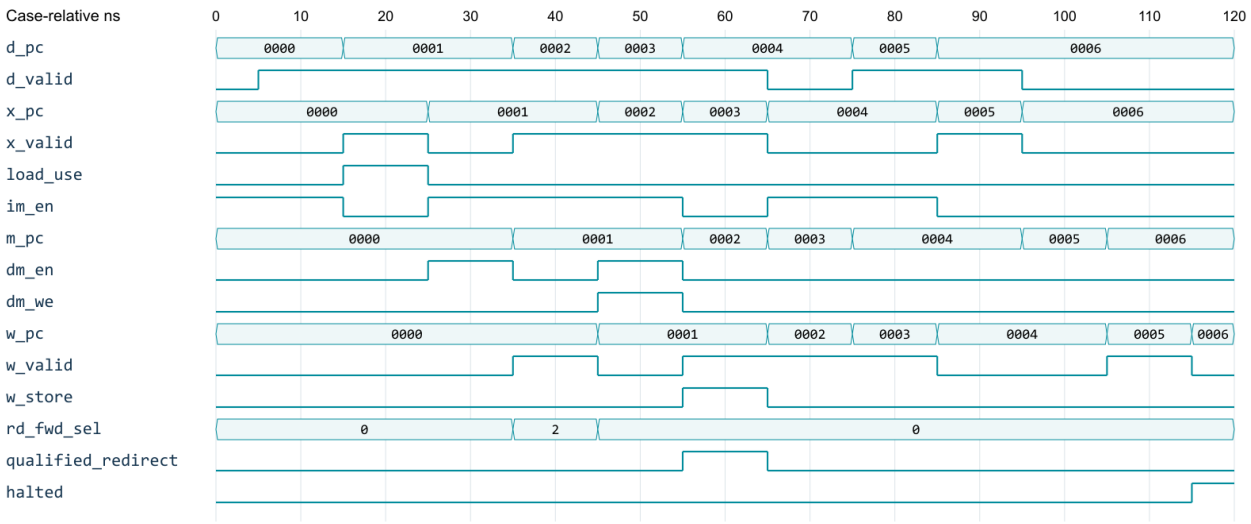


图 12 冻结 v1.6 历史 load→STR 波形。AC1 对应 CSV 保持相同：C3 保持、C4 bubble、C5 从 W 前递；STR 之后保留有效 WB。原图身份保留，不冒充本次文档同步的新仿真。

样例机器字 8300 9100 A000 D001 F000 E000；R1=0021，R3=0020，DM[0020]=0007。C3 的 load_use 禁止取指并保持 ID；C5 的 STR 从 WB load 取得 0007，C6 在 MEM 写 DM[0021]，C7 仍有有效 STR WB 且无写入。JZ 取消年轻 F000，C12 HALT WB 结束后停止。横轴为用例相对 ns，Ck 结束边沿在 5+10×(k−1)ns；不要把同列 D 机器字和 X pc 当作同一条指令。

冻结 v1.6 初次记录 6a4df640 保留原身份。AC1 原记录 72d99981 与核心包内 job1 记录 eae32d52 各自绑定 13RTL、TB、脚本、镜像及 19 步执行；后者 9 份 CSV 与前者逐字节一致。两次均实际通过 2,550 冒险案例和 131,072 前递控制组合，后者仅使用 4 个数据哨兵，非全 16 位数据穷举。独立 48 例含 952 逐拍行、四个文档例及 10 复位见证，核心包内记录为验证记录/cpu_ac1_independent_dpgqkqum。完整 UUID 及路径见证据索引.json；功能 PASS 不代替板级时序。

4.5.4 覆盖范围与集成验收

验收包括单元输入空间、流水时序和指令执行三个层面。单元回归确认模块功能，流水测试检查跨级行为，指令覆盖则确认当前 ISA 从译码到架构写入的完整路径。

4.5.4.1 单元输入与状态覆盖

表 41 单元模块的精确覆盖范围

对象	输入或状态范围	验收重点
CLA4	a4 b4 cin1 共 512 组	sum cout P/G 及 cin 独立性
CLA16 独立单元	a16 b16 cin1 全量为 2^33	结果与进位 长进位 组间超前结构

对象	输入或状态范围	验收重点
AC1 ADD/SUB 核心	a16 b16 sub1 全量为 2^33	每种运算 2^32；result/C/V 均检查
ALU	a16 b16 op3 全二进制空间 2^35	5 合法及 3 非法 op eq 独立 非算术 C/V=0
新译码器	全部 65,536 指令字	两张控制表 源/use 选择 扩展与副作用
扩展器	imm8 mode1 共 512 组	零扩展 符号扩展
flags_reg	8 旧状态×2rst×4 使能×8 输入 共 512 组	复位 保持 独立字段及同时写

课堂要求全量的对象按精确输入空间执行，不能把单 op 的 2^32、通用 cin 测试的 2^33 和全部 ALU 选择的 2^35 混称同一种全量。长回归按连续区间计数，记录失败反例和最终完成数，关闭逐向量波形/CSV；没有完成日志不写全量通过。

4.5.4.2 流水相关与停止测试

下表每例独立复位，IM 从 0000 装入机器字，IM/DM 各 256 字。初始 RF 由 TB 在第一条取指前预置；这不是 ISA 增加 16 位立即数指令。未列寄存器、Z/C/V 及 DM 默认 0。下列定向用例已在本版 RTL 回归中执行；表中保留其验收预期，实际结果见对应 CASE_PASS 和时序见证。

表 42 流水相关与停止定向用例

用例	初始条件与机器字序列	预期状态与时序
T01 CMP 后算术	R0=0001 R1=0001 R2=FFFF；A100 5801 D001 1C55 E000	ADD 后 R2=0000 C=1 V=0；Z 仍 1；JZ 跳过 MOV R3
T02 连续 CMP	R0=0001 R1=0002；A000 A100 D001 1855 E000	最近 CMP 输出 0 覆盖旧 1；JZ 不跳；R2=0055
T03 地址运算保持标志	R0=7FFF R1=0020 DM[001F]=BEEF；5001 89FF 9900 E000	R0=8000 C=0 V=1；R2 及 DM[0020]=BEEF；访存保持 C/V；load 到 STR 数据停 1 拍
T04 错误路径写入	R0=FFFF；C003 5001 A000 E000	ADD/CMP 均被取消；RF 和 Z/C/V 保持
T05 老 WB 与年轻故障	R0=7FFF R1=00FF；5001 8901 5801 E000	ADD 在 WB 与 LDR 在 MEM 越界同拍；R0=8000 C=0 V=1；故障 load 和年轻 ADD 无写入
T06 load use 与 CMP	R1=0020 R2=0001 DM[0020]=0001；8100 A200 D001 1C55 E000	load 到 CMP 停 1 拍；R0=1 Z=1；JZ 跳过 MOV R3

用例	初始条件与机器字序列	预期状态与时序
T07 HALT 排空	R0=0020 DM[0020]=1234；8400 E000 9401	老 LDR 写 R1=1234；HALT 后 STR 取消，DM[0021]保持 0
T08 停顿不冻结 WB	R0=7FFF R1=0020 DM[0020]=0001；5001 1C00 8900 4A00 E000	load 到 ADD R2,R2 停 1 拍；同拍老 ADD 写 C=0 V=1；末尾 R2=2 C=0 V=0

T09 对应 tb_datapath.sv 的 reset_probe，共 16 次。正常序列为 1020 1405 9400 8800 A200 E000，以 phase=0 至 12 调用；故障序列为 1001 B000 E000，以 phase=4 至 6 调用。TB 在下降沿断言 rst 并跨越下一上升沿，检查 RF、Z/C/V、PC、valid 和停止状态清 0，DM 全部保持。正常序列 phase=5 使待写 STR 遇到复位而禁止写入，phase=6 确认已经完成的 STR 不被清除；故障序列覆盖 DRAIN_FAULT 和 FAULTED 复位。随后另一次用例准备再撤销 rst；独立 audit3 另有 4 次停止边沿或停止后复位并重启的测试。T09 是时序测试，不能用顺序指令模型替代。

集成测试同时检查最终状态和逐拍动作。T01 检查 CMP 后 ADD 保持 Z；T05 检查年轻 MEM 故障同拍老 WB 仍更新；T08 检查前端 stall 同拍老 WB 仍完成。本次主回归检查必需时序见证是否实际出现，缺少见证即失败；文档级顺序指令模型仍只用于核对机器字与架构预期，不代替这些 RTL 记录。

flags_reg 的 512 组覆盖采用“建立旧状态，再执行一次计数转移”的方法，建立状态的时钟不计入 512。CU 测试覆盖图 8 的可达转移、RUN 下停顿/重定向、DRAIN 期间较老写入、停止保持和各状态复位。T07 断言 HALT 在 MEM 时 halted 仍为 0，在有效 HALT 的 WB 结束边沿置 1，随后所有 valid 为 0 且停止期间没有新访问。T16 检查 MEM 故障与 HALT 同拍时直接进入 DRAIN_FAULT，不应经过 DRAIN_HALT 或 HALTED。DRAIN_HALT 中不得出现新的已确认故障，否则 TB 报告时序契约被违反。故障测试对 FAULTED 采用相同的 WB 标记断言。测试设超时退出，避免错误控制导致无限等待。

T17a/T17b 在故障 STR 的 MEM 周期检查完整 EA、dm_en=0 和 dm_we=0；逐字比较 DM，确认没有按低 8 位误写 0000 或 00FF，同时检查老 WB 完成及年轻操作取消。该故障 STR 在下一拍保留有效 WB 故障标记，结束边沿进入 FAULTED。T18a 对越界取指检查 im_en=0、IF/ID 的 pc=0100 和 fetch_fault=1；故障沿有效载荷传到 EX 确认，再于 WB 完成停止。T18b 检查更老分支取消年轻取指故障，目标 HALT 正常结束。状态断言在采样边沿的非阻塞赋值更新完成后执行；DM/IM 使能在边沿前采样。

4.5.4.3 全部指令的执行覆盖

以下用例验证 MOV 寄存器形式、SUB 寄存器形式、AND 和 OR，并检查正确路径上的保留编码以及 JZ 负位移。每个子例独立复位，沿用前述 256 字 IM/DM 和 RF 预置规则；未列状态为 0。正常程序未使用的 IM 字填 B000。表中列出验收预期，本次对应 CASE_PASS 与时序检查均已完成；后续修改仍须复跑。

表 43 指令执行与故障定向用例

用例	初始条件与机器字序列	预期状态与时序
T10 MOV 寄存器	R0=55AA；Z=C=V=1；140F 0100 E000	R1 及 R0=000F；Z/C/V 保持 1；紧邻 MOV 使用 Rs 转发，不读取旧 Rd
T11a SUB 无借位及溢出	R0=8000 R1=0001 Z=1；6100 E000	R0=7FFF C=1 V=1 Z=1；R1 保持 0001
T11b SUB 借位	R0=0000 R1=0001 Z=1；6100 E000	R0=FFFF C=0 V=0 Z=1；不能把 borrow 直接当作 C
T12 AND	R0=A5A5 R1=0F0F；Z=C=V=1；2100 E000	R0=0505 R1=0F0F；Z/C/V 保持 1，不被非算术 C/V 候选 0 覆盖
T13 OR	R0=A500 R1=00A5；Z=C=V=1；3100 E000	R0=A5A5 R1=00A5；Z/C/V 保持 1
T14a 保留 B 编码	1001 B000 1402 E000	老 MOV 完成，R0=0001；年轻 MOV/HALT 取消；R1=0 fault=1 halted=0
T14b 保留 F 编码	1001 F000 1402 E000	与 T14a 相同；故障标记在 WB 消费前 fault 仍为 0
T15 JZ 负位移	1002 1401 7001 A100 D0FD E000	pc=0004 的目标为 0002；第一次跳，第二次不跳；末尾 R0=0 R1=1 Z=0 C=1 V=0
T16 MEM 故障优先于 HALT	R1=00FF R2=55AA；8901 E000 1001	LDR 越界同拍取消 EX 的 HALT；R2 保持 55AA R0=0；fault=1 halted=0
T17a STR 上界越界	R0=7FFF R1=00FF Z=1 DM[0000]=CAFE DM[00FF]=BEEF；5001 9101 1C55 E000	EA=0100；DM 全部保持；老 ADD 写 R0=8000 C=0 V=1 Z=1；年轻 MOV 取消；fault=1 halted=0
T17b STR 负位移越界	R1=0000 DM[00FF]=BEEF；1001 91FF E000	EA=FFFF；DM 全部保持；老 MOV 写 R0=0001；HALT 取消；fault=1 halted=0

用例	初始条件与机器字序列	预期状态与时序
T18a 正确路径取指越界	IM[0000]=C0FF IM[00FF]=1001；其余 IM 填 B000	跳至 00FF，MOV 写 R0=0001；下一取指 0100 产生故障；fault=1 halted=0；DM 保持
T18b 分支取消取指故障	IM[0000]=C0FF IM[00FF]=C002 IM[0002]=E000；其余 IM 填 B000	00FF 的 JUMP 取消年轻 0100 取指故障，转至 0002；HALT 正常结束；RF/DM 保持 0，fault=0 halted=1

T18a/T18b 按列明的字地址装入稀疏 IM 镜像，不把机器字依次放在 0000 起的连续地址；两例均从正常同步复位状态开始。T18b 已通过年轻 0100 取指故障取消及不得进入有效 EX 的见证检查。其余连续序列按前述从 0000 装入。本次结论来自迁移后的真实 RTL 复跑；顺序指令模型只校对编码与架构预期，不代替硬件时序检查。

下表把 14 种合法 Opcode 映射到 CPU 级执行用例。它检查每种编码是否实际执行、使用正确的数据源并写入正确目标；单元 ALU 运算覆盖和 65,536 字译码覆盖不能代替这张表。基本用例通过后，再系统改变 Rd/Rs 索引、立即数边界、RAW 距离及初始标志，执行指令级差分回归。

表 44 全部合法指令的 CPU 执行覆盖

指令编码	指令形式	CPU 执行用例	必须检查
0	MOV Rd,Rs	T10	PASS_B 与 Rs 选择；Rd 写回；标志保持
1	MOV Rd,#u8	T10 及功能程序	零扩展；不使用旧 Rd；标志保持
2	AND	T12	按位结果；只写 RF；Z/C/V 保持
3	OR	T13	按位结果；只写 RF；Z/C/V 保持
4	ADD Rd,Rs	T08 及功能程序	两源转发；RF 与 C/V 写回；Z 保持
5	ADD Rd,#u8	T01 T03 T08	零扩展；C/V 边界及 Z 保持
6	SUB Rd,Rs	T11a T11b	C 为无借位；V 独立；Z 保持
7	SUB Rd,#u8	T15 及功能程序	零扩展；低 16 位结果与 C/V
8	LDR	T03 T05 T06 T07	符号位移；DM 读及 WB；越界禁止写回
9	STR	T03 及功能程序；T17a/T17b	合法写入；越界禁止 DM 访问；故障标记保留
A	CMP	T01 T02 T06	只写 Z；最近 CMP 优先；不写 RF/C/V

指令编码	指令形式	CPU 执行用例	必须检查
C	JUMP #u8	T04 T18a T18b	绝对目标；取指边界；年轻副作用取消
D	JZ #s8	T01 T02 T15	跳与不跳；负位移；使用指令 pc 标签
E	HALT	T07 及功能程序	老操作完成；年轻操作取消；WB 结束标记

B/F 不是合法指令，分别由 T14a/T14b 检查正确路径故障；功能程序另检查 HALT 取消年轻保留编码。覆盖计数只在有效指令完成其规定动作时更新，取到或译码到一条最终被取消的指令不算该指令已执行。除目标 RF/DM/标志外，TB 还要检查未指定写入的寄存器、标志及 DM 保持不变，并对每次实际副作用核对 pc 标签、目的和数据。

4.5.5 CPU 功能验证程序

下面程序从正常复位状态运行，使用 MOV 建立输入，不依赖 TB 修改 RF 内部状态。它覆盖相邻算术相关、存储与加载、load 到 CMP、Z 转发、错误路径取消及 HALT 结束。

表 45 CPU 功能与性能基准程序

字地址	机器字	指令	预期动作
0000	1005	MOV R0,#5	R0=0005
0001	1403	MOV R1,#3	R1=0003
0002	4100	ADD R0,R1	转发源值；R0=0008 C=0 V=0
0003	7401	SUB R1,#1	R1=0002 C=1 V=0
0004	1C20	MOV R3,#32	R3=0020
0005	9300	STR R0,[R3+0]	DM[0020]=0008
0006	8B00	LDR R2,[R3+0]	R2=0008
0007	A200	CMP R0,R2	load use 停 1 拍；Z=1
0008	D001	JZ +1	目标 000A；使用 CMP 候选 Z
0009	14FF	MOV R1,#255	错误路径；不得执行
000A	E000	HALT	排空后 halted=1

最终应为 R0=0008、R1=0002、R2=0008、R3=0020、DM[0020]=0008、Z=1、C=1、V=0、fault=0。11 个静态机器字中有 10 条有效动态指令。程序同时验证 STR 后同地址 LDR 读取新值，不能只检查 RF 结果而遗漏 DM 副作用。

本测试的其余 IM 字填 B000，DM 初始填 0。HALT 到 EX 确认前可能已取到其后的保留编码；这些年轻故障标记应被 HALT 取消，不能一到 ID 就设置全局 fault。停止程序的正确结果仍为 halted=1、fault=0。

4.6 性能基线与统计口径

先明确测量对象，再比较周期与实现资源。历史 core-only、文件启动系统 OOC、含 VIO 的完整板级实现具有不同端口和观察开销。下述固定工作负载在同一程序、RAM、调试配置、器件及工具约束下成对比较。

完整布局布线在 100MHz 约束下通过，不等于实物已运行；程序纳秒数按周期×10ns 计算，物理测量仍未完成。

4.6.1 CPU 性能验证

功能通过后，在相同程序、初值、存储配置和工具约束下测量性能。C 从第一个有效 IF 周期开始，计至 HALT 的 WB 停止标记消费边沿，包含填充、停顿和冲刷，排除复位准备；N 统计有效动态指令，包含 STR、CMP、分支和 HALT，不包括 bubble 或错误路径。同拍可观察老 WB 完成与年轻 STR 在 MEM 写入，但 STR 的统一完成计数在后一拍 WB。原架构副作用模型仍检查 WB 后 MEM 的年龄次序；新增 WB 完成模型单独检查每条标签，禁止重复计数。

```
CPI = C / N
IPC = N / C
T_program = C / f
MIPS = f / (CPI * 1000000.0)     // f in Hz
```

阶段 2 文件启动 baseline 再次得到 17 周期、10 个有效动态指令、1 次 load-use 和 1 次 taken 分支；all_ops 得到 25 周期、16 条动态指令、1 次停顿和 2 次重定向，综合网表回放相同。基准计数含填充、排空、有效 STR 与 HALT，排除复位准备；不把 OOC 估算直接换算为已实现时钟。

TB 统计 C/N 时保留足够位宽的整数计数器，计算 CPI、IPC 和程序时间时转为实数，避免整数除法截断。上述是度量关系，不是需要在 CPU 中综合的除法电路。

报告分别给出 C/N、load use 停顿数、取分支次数、实际工作时钟以及 LUT/FF/BRAM/CARRY 资源。程序时间 T 是比较结果，CPI 和 f 应一起说明；综合资源或理想 IPC=1 不等于 CPU 性能验证完成。Fmax 只在约束、布局布线和完整 setup/hold 检查范围内讨论，行为仿真的#1 延迟不用于推导 Fmax。

表 46 三种资源和时序统计边界

对象	LUT / FF / BRAM18	时序与适用范围
历史 core-only AC1	287 / 260 / 0	历史 OOC WNS +2.620ns；观测扩展前，不能当整机资源

对象	LUT / FF / BRAM18	时序与适用范围
文件启动 cpu_system	376 / 431 / 2	已存系统 OOC WNS +1.363ns；含 IM/DM 和影子
含 VIO 板级 AC1	五程序 LUT1082 ~ 1160；FF2822 ~ 2826；BRAM18 均 2	完整布线 setup WNS +1.434 ~ +2.101ns；不是实物时钟或时间测量

4.6.2 固定程序与性能基线

表 47 五个工作负载的完整实现基线

工作负载	周期 / N	stall / 跳转	LUT	FF	setup / hold ns
all_ops	25 / 16	1 / 2	1118	2826	2.086 / 0.071
baseline	17 / 10	1 / 1	1120	2825	1.838 / 0.083
loop	30 / 18	0 / 4	1160	2822	2.101 / 0.062
memory_copy	15 / 7	2 / 1	1119	2823	1.540 / 0.086
copy_checksum	55 / 35	16 / 0	1082	2824	1.434 / 0.073

五个配置都使用 2 个 BRAM18；面积采用 routed utilization 的 Total LUTs，包含 LUTRAM。每个程序的 98 项共同输入相同，选定镜像和独立 gold 数据是明确变量。程序不同可能导致常量优化及资源差异，候选比较必须使用同一程序成对评价。

复制校验程序包含 35 条动态指令：8 次 load/store 复制，再读回 8 个目的字并累加。初值 FFFF、0001、8000、7FFF、1234、A5A5、5A5A、0000，最终 R1=1232，R0/R2=0，R3=32；最后 store 地址 71、数据 0 且 valid=1。AC1 实际为 55 周期、16 停顿，避免仅用最后一个复制字证明整段程序。

按已实现 10ns 时钟计算，五个程序时间为 250、170、300、150、550ns。它们是执行周期推算，不是尚未进行的板上测量，也不含复位去抖、配置和 JTAG 等待。实际 CPU 时钟必须与布局布线配置和板卡时钟相符。

4.7 隔离性能候选与取舍

代码组织与微架构优化使用不同验收条件。前者应保持原每拍行为；WB 到 MEM 改变特定 STR 数据相关的停顿规则，必须保持 ISA 事件顺序并通过自己的新周期契约。两个候选不叠加，默认执行核心继续使用 AC1。

copy_checksum 正常 HALT 程序包含 35 条完成指令。默认 AC1 为 35+4+16=55 拍；WB 到 MEM 候选为 35+4+8=47 拍。收益来自 8 次 STR 数据相关停顿的消除，不是额外提高频率，也不适合作为 fault/reset 程序的通用周期公式。

4.7.1 前递 MUX 组合候选

forward_onehot 只替换 operand_forward 的数据 MUX 表达式；原 if 控制判定及 sel/blocked 语义保持，M 匹配 load 仍阻断旧 W 值，R0 参与普通相关比较。原九套、独立 48 例及错误注入通过，五个程序完整公开逐拍轨迹一致。掩码 OR 不保证选中 Z 载荷的四态数据等价，本轮功能结论限于已执行的输入与轨迹。

表 48 前递 MUX 候选相对 AC1

程序	LUT 变化	setup 裕量变化 ns	100MHz 周期变化
all_ops	21	+0.136	0
baseline	13	-0.148	0
loop	-20	-0.488	0
memory_copy	15	-0.265	0
copy_checksum	108	-0.426	0

该候选在 4 个负载中增加 LUT，在 4 个负载中降低 setup 裕量；固定 100MHz 下没有周期收益。本轮保持稳定 AC1，不把改写 MUX 本身当作优化成功。

4.7.2 WB 到 MEM 存储数据候选

wb_mem 在 EX 不消费尚未就绪的 STR 数据，只把必要资格与占位载荷传至 MEM。优先复用 m_rd 表示 STR 数据源，增加一位 m_store_pending；MEM 检查当前 WB 是否为同名、有效、无故障、可写的 load 生产者，再选择已就绪 wb_data。地址仍在 EX 使用可用 Rs 计算，基址相关保留一拍停顿。

EX 例外必须匹配合法 STR 控制；未知或非法资格不能绕过 blocked 检查。MEM 未就绪时禁止写 DM 并发内部故障原因 5，地址越界保持原因 4；较老 MEM 事件优先年轻 EX，老 WB 继续完成。STR 在 MEM 写一次，其有效标签继续 WB，无 RW/ZW/CVW。reset、取消和 bubble 均不能保留可产生副作用的 pending。

表 49 WB 到 MEM 候选的实际周期比较

程序	AC1 周期 / stall	WB 候选周期 / stall	周期推算差值 ns
all_ops	25 / 1	25 / 1	0
baseline	17 / 1	17 / 1	0
loop	30 / 0	30 / 0	0
memory_copy	15 / 2	14 / 1	-10
copy_checksum	55 / 16	47 / 8	-80

该候选已通过 48 个原 ISA 用例和 10 个真实复位见证，以及 2005 个候选用例（2002 个正常 HALT、3 个故障，其中 64 个为固定种子压力程序）。逐边沿检查 RF、DM、标志、指令顺序、STR 到 WB 标签和停止状态；正常 HALT 程序另检查新周期与停顿数。另有 12 项内部协议/复位保护检查和 6 个被明确捕获的错误变体，包括非法和 X 控制不能借用 EX 延迟数据例外。此为有限仿真，不等于全状态形式证明。

原冻结 48 例保持原字节；私有执行 TB 仅排除两条旧版总周期/停顿期待，架构断言、复位见证及新控制检查继续生效。新候选不要求旧时序 CSV 逐字节相同，而要求架构事件与候选周期契约同时成立。故障/reset 不机械套正常 HALT 公式：年轻 STR 减少停顿，不一定缩短较老 fault 自身排空时间。

表 50 WB 到 MEM 候选的完整实现代价

程序	LUT 变化	FF 变化	setup 裕量变化 ns
all_ops	-2	2	-0.303
baseline	85	2	-0.099
loop	-50	2	-0.566
memory_copy	84	2	-0.047
copy_checksum	86	2	+0.234

WB 到 MEM 在访存数据相关负载上减少执行周期，但额外资格检查、pending 寄存器和数据选择也有实现代价。当前保持为独立实验候选，默认 AC1 源码不替换；板上结果尚未确认。实测频率或存储时延改变时，不能直接照搬本轮固定 100MHz 的时间差。

4.8 重现入口与验收边界

文档的当前维护入口为文档源/build_core_document.py。architecture.json 定义章节、职责和 CW；sections.json 保存已迁移的成熟段落、表格与稳定编号锚点；content.json 保存新增说明。新稿不依赖原 Word 段落下标，旧生成脚本仅用于历史复现并禁止覆盖本架构版。

当前物理验证按用户要求暂缓；本文没有真实板上功能、按键行为或直接运行时间测量的 PASS 结论。所有板卡连接与烧录均留待用户明确恢复上板任务后执行。

```
python -B -X utf8 CPU 核心设计/Vivado/run_independent.py
python -B -X utf8 CPU 核心设计/文档源/build_core_document.py --independent-run <新 48 例记录>
python -B -X utf8 CPU 核心设计/文档源/validate_document.py --document <核心 Word>
```

综合检查锁存器、组合环和未连接端口；布局布线后检查 setup/hold、适用的脉宽/复位要求、DRC 及未约束路径。单周期 EX 路径不能靠 false path 或 multicycle 例外隐藏失败。ALU 速度比较采用相同器件、相同寄存器边界和约束，手工 CLA 结构不自动保证 FPGA 实现更快。

AC1 的有限功能接受覆盖合法/故障 Opcode、地址保护、年轻取消、停顿/前递、WB 停止边沿和同步复位。本版另有完整板级实现 STA；未完成物理板测、全状态形式证明或完整算术穷举。可变延迟存储未实现，任何局部 PASS 均不扩大为这些范围的通过。

证据索引.json 分别保留历史来源和本轮新执行。源码变化必须重跑受影响验证并绑定新指纹；只有历史校验不能生成 fresh XSim 结论。系统 run_system.py 的 PASS 表示所选有限检查完成，timing_closed、placed_and_routed、board_validated 仍为 false。

4.9 参考资料

技术参考采用 AMD UG474 的进位资源说明和 AMD UG903 的时序约束规则。文档组织参考 Ibex Reference Guide 与 CORE-V Core Integration 中的模块、接口和流水描述方法；这些参考不改变本 CPU 的自定义 ISA 及五级实现。

课程方法依据包括 CHA Lec 01-B 的 ISA 到数据通路/CU 流程、CHA Lec07-Sequential circuits analysis and design 的 F/Q/G 及状态转移方法，以及第 7 次课.zip 中的 Lec18 RISC CPU Design-CU design。加减器、HDL、Testbench、性能与 ISA/数据通路内容继续参考课程对应资料及《数字系统设计》第二版第 4 章、第 7 至 8 章。课程串行实验 FSM 用于理解方法，本 CPU 采用本文规定的五级并行、有效位与模式状态机。

当前 ISA 依据为 2026 年 10 月 11 日确认的 Group 1 至 Group 7 编码表，与最终 Pre4 汇报第 3 页的 Opcode 分组一致。使用其字段划分与控制副作用规则；机器字示例以第一章按位校核后的结果为准。

板级器件、PL Bank 电平、时钟、按键及 LED 引脚依据 EES331 User Manual v1.0 1220 的 PDF 第 5、7、29 至 30 页；实现工具和约束以 Vivado2025.2 原始报告为准。EES-331 使用 CLG484 封装，本版不采用 PYNQ-Z2 的 CLG400 封装或 125MHz 时钟配置。